

Rapport de stage

Benchmarking interactif, développement de scRAW et évaluation inductive pour le clustering scRNA-seq

Fabien Bidet

Avril 2026

Période de stage : Janvier-Juin 2026

Organisme d'accueil : LaBRI (Laboratoire Bordelais de Recherche en Informatique)

Responsables de stage :

- **Adélaïde Raguin** – Chargée de recherche CNRS, chercheuse au LaBRI (Tutrice de stage)
- **Victoria Bourgeois** – Maître de Conférences, chercheuse au LaBRI, Université de Bordeaux (Encadrante universitaire)
- **Loann Giovannangeli** – Maître de Conférences, chercheur au LaBRI, Université de Bordeaux (Encadrant universitaire)
- **Patricia Thébault** – Professeure des Universités, chercheuse au LaBRI, Université de Bordeaux (Encadrante universitaire)

Table des matières

1	Introduction	3
1.1	Environnement de travail	3
1.2	Introduction du sujet de stage	4
2	État de l’art	5
2.1	Données	5
2.2	Traitements de données	7
2.2.1	Pré-process (Le nettoyage initial)	7
2.2.2	Batch effect (L’effet de lot)	7
2.3	Algorithmes de clustering	8
2.3.1	Approches classiques	8
2.3.2	Approches Deep Learning (Apprentissage Profond)	8
2.3.3	Les types cellulaires dans les algorithmes	10
3	Comparaison de l’existant	11
3.1	Logiciel	12
3.2	Expériences	14
3.2.1	Données et splits	14
3.2.2	Méthodes	16
3.2.3	Résultats	18
4	scRAW	19
4.1	Positionnement et motivation	19
4.2	Pipeline	20
4.3	Fonctions de perte et variantes	21
4.4	Recherche d’hyperparamètres	22
4.5	Résultats et discussion	24
5	Travaux complémentaires	24
5.0	Transfert de loss	24
5.1	Induction	24
5.2	Interprétation biologique ?	25
	Conclusion finale à rédiger	25
	Figures et tableaux restants à préparer	25
A	Structure et organigramme du LaBRI	26
B	Détails des datasets de l’état de l’art	28
C	Tableau des métriques d’évaluation	29
D	Hyperparamètres de scRAW <code>stable_default</code>	30

E	Figures de recherche d'hyperparamètres scRAW	33
F	Screenshots de l'interface graphique SCRBenchmark	36

1 Introduction

1.1 Environnement de travail

Ce stage de fin d'études s'est déroulé du 4 janvier au 30 juin 2026 au sein du Laboratoire Bordelais de Recherche en Informatique (LaBRI). J'ai intégré l'équipe Bench to Knowledge and Beyond (BKB), dirigée par Romain Bourqui (responsable d'équipe) et Patricia Thébault (adjointe). Cette équipe est rattachée au département Systèmes et Données (SeD). Afin de mieux appréhender la structure dans laquelle j'ai évolué, l'organisation détaillée du laboratoire ainsi que son organigramme structurel sont présentés en Annexe A (Figure 5).

Équipe d'encadrement

Au cours de ce stage, j'ai été encadré par une équipe aux expertises complémentaires :

- **Victoria Bourgeais**, enseignante-chercheuse au LaBRI, experte dans l'application de l'apprentissage profond au domaine de la santé ;
- **Loann Giovannangeli**, enseignant-chercheur au LaBRI, en visualisation de l'information et intelligence artificielle ;
- **Patricia Thébault**, enseignante-chercheuse au LaBRI, en sciences des données omiques et réseaux biologiques.

Cadre de travail

J'ai effectué mes travaux dans un bureau partagé avec des doctorants et d'autres stagiaires. Ce contexte s'est révélé particulièrement enrichissant, facilitant les échanges de connaissances au quotidien et me permettant de mieux cerner les enjeux d'une thèse et de la vie de jeune chercheur(se).

Par ailleurs, j'ai pu assister aux séminaires hebdomadaires organisés par les doctorant(e)s de l'équipe BKB, durant lesquels des chercheurs invités présentaient leurs travaux en bioinformatique. J'ai également eu l'opportunité de participer aux séminaires d'autres équipes, notamment celles spécialisées en intelligence artificielle. Ces événements ont constitué une source d'apprentissage scientifique de pointe, idéale pour mon profil situé à l'interface entre la bioinformatique et l'intelligence artificielle. D'un point de vue technique, j'ai bénéficié d'un accès distant (via SSH) à un serveur de calcul interne au LaBRI pour réaliser mes expérimentations.

Suivi et méthodologie

L'avancement de mes travaux a été rythmé par des réunions régulières. Plusieurs réunions mensuelles étaient organisées avec mes encadrants pour faire le point sur l'avancement scientifique. De plus, des réunions bimensuelles ont été planifiées avec Elodie Darbot et Cyril Dourthe, deux bioinformaticien(ne)s et chercheur en bioinformatique, et plusieurs stagiaires, afin de discuter et de présenter nos avancements dans nos projets. Enfin, j'ai pu compter sur la disponibilité de ma tutrice de stage Adélaïde Raguin, chercheuse au LaBRI, pour faire le point sur mon avancement et répondre à mes interrogations tout au long de cette période.

1.2 Introduction du sujet de stage

Le séquençage de l'ARN à l'échelle de la cellule unique (*single-cell RNA sequencing*, scRNA-seq) a profondément transformé l'analyse des tissus biologiques. Contrairement aux approches transcriptomiques classiques, comme l'analyse bulk-ARN, qui mesurent une expression moyenne sur un ensemble de cellules, le scRNA-seq permet d'observer les profils d'expression cellule par cellule. Il devient alors possible d'étudier l'hétérogénéité interne d'un tissu, d'identifier différents types ou états cellulaires, et de mettre en évidence des populations minoritaires qui auraient été masquées dans une mesure globale [1, 2, 3, 4, 5].

Cette richesse d'information s'accompagne cependant de difficultés importantes. Les données scRNA-seq sont de très grande dimension, car chaque cellule est décrite par l'expression de plusieurs milliers de gènes. Elles sont également bruitées, très creuses, sensibles aux événements de non-détection (effet dropout) et souvent affectées par des effets de batch liés au protocole expérimental, au laboratoire ou à la plateforme de séquençage [6]. Avant toute interprétation biologique, il est donc nécessaire de construire une représentation plus compacte et plus robuste des cellules.

Dans ce contexte, le clustering constitue une étape centrale des analyses scRNA-seq. Il consiste à regrouper les cellules présentant des profils d'expression similaires afin de retrouver des types cellulaires, des sous-types ou des états fonctionnels. Des pipelines classiques, comme ceux fondés sur une réduction de dimension suivie d'un clustering, sont aujourd'hui largement utilisés dans les outils bioinformatiques standards [7, 8]. Plus récemment, les méthodes d'apprentissage profond ont cherché à apprendre des espaces latents plus adaptés à la complexité des données, notamment à l'aide d'autoencodeurs ou de modèles probabilistes [9].

Un des enjeux abordés pendant ce stage concerne plus spécifiquement les populations cellulaires rares. En biologie, ces populations peuvent correspondre à des cellules immunitaires minoritaires, à un sous-type tumoral, à une population pathologique, à un état transitoire de différenciation ou encore à des cellules spécifiques d'un tissu. Leur faible proportion ne signifie donc pas qu'elles sont secondaires : au contraire, elles peuvent porter un signal biologique essentiel. Pourtant, du point de vue algorithmique, elles sont particulièrement fragiles. Comme elles représentent peu d'observations, elles contribuent faiblement à l'apprentissage des modèles et peuvent facilement être absorbées et confondues par une population majoritaire ou par un type cellulaire proche. Un bon score moyen dans une métrique généraliste peut alors masquer un échec important : la disparition d'une population biologiquement pertinente.

L'enjeu de ce mémoire est alors de développer une solution unique qui permettrait à la fois d'obtenir un clustering global précis, tout en permettant l'identification des cellules rares et ce même face à des jeux de données comportant un bruit technique important et de pouvoir utiliser ce modèle sur de nouvelles données non vues pendant l'entraînement. Ce dernier point est particulièrement important : dans un contexte réel, il peut être pertinent d'utiliser un modèle déjà entraîné car cette tâche peut être très coûteuse si il faut la répéter à chaque fois que l'on veut utiliser notre modèle. Il doit pouvoir projeter ou classer de nouvelles cellules, issues par exemple d'un nouveau patient, d'un nouveau batch ou d'une expérience complémentaire. On distinguera alors les approches transductives, qui travaillent uniquement

avec les données disponibles, et inductives, qui peuvent généraliser à de nouvelles données.

Mon stage s'est inscrit dans cette problématique à l'interface entre bioinformatique, apprentissage automatique et évaluation expérimentale. Le travail réalisé s'est articulé autour de quatre contributions principales. La première a consisté à développer et structurer **SCRBenchmark**, une interface web interactive destinée à comparer différentes méthodes de clustering scRNA-seq sur plusieurs jeux de données. La deuxième a porté sur la comparaison de méthodes existantes, classiques et fondées sur l'apprentissage profond, en intégrant des métriques globales mais aussi des métriques centrées sur les classes rares. La troisième contribution a été le développement de **scRAW**, une méthode visant à apprendre une représentation latente en donnant davantage d'importance aux cellules rares ou peu denses pendant l'apprentissage. Enfin, des travaux complémentaires ont exploré le transfert de cette logique de pondération vers d'autres pertes et les premiers tests d'un comportement inductif.

La question directrice du mémoire peut ainsi se formuler de la manière suivante : comment construire, évaluer et améliorer un pipeline de clustering scRNA-seq capable de mieux préserver les populations cellulaires rares, tout en restant reproductible et utilisable sur de nouvelles données ? Pour y répondre, la suite du rapport présente d'abord les notions nécessaires à la compréhension des données scRNA-seq et des méthodes de clustering. Elle décrit ensuite le protocole de comparaison mis en place, puis détaille la conception et l'évaluation de scRAW. Enfin, elle discute les extensions explorées pendant le stage, notamment le transfert de loss, l'induction et les pistes d'interprétation biologique.

2 État de l'art

L'objectif de cette section est de présenter les bases nécessaires à la compréhension du rapport pour mieux comprendre mes contributions et où elles se situent par rapport à la recherche actuelle. Nous commencerons par examiner les jeux de données utilisés, puis nous verrons comment ces données sont préparées. Enfin, nous ferons une présentation des algorithmes de clustering existants, en comprenant notamment pourquoi les modèles d'intelligence artificielle actuels ont souvent du mal à repérer les cellules rares.

2.1 Données

Pour qu'une méthode de clustering soit considérée comme robuste, elle doit faire ses preuves sur des données variées. Dans le cadre de ce stage, nous nous sommes appuyés sur une collection de jeux de données diversifiés (détaillés dans nos tables de référence).

Ces données proviennent de différents organes et espèces : pancréas humain et de souris, cerveau, rate, sang, testicules, foie, ou encore rétine de macaque. Chaque jeu de données possède ses propres défis :

- **La taille :** Cela va de petits jeux de données d'environ 2 000 cellules (comme *Paul15 bone marrow* ou certains sous-jeux du pancréas) jusqu'à de très gros ensembles dépassant les 30 000 cellules (comme la rétine de macaque ou les PBMC).
- **Les effets de batch :** La plupart des jeux de données sont composés de plusieurs "batches" (lots expérimentaux). Par exemple, les données de PBMC combinent 16 lots différents,

et la rétine de macaque en compte 30. D'autres jeux de données peuvent combiner plusieurs échelles d'effet batch, en mélangeant par exemple différentes technologies de séquençage et plusieurs patients.

- **Les populations rares :** C'est le cœur de notre problématique. Presque tous nos jeux de données contiennent des types cellulaires dits "rares" (représentant moins de 5 % des cellules) ou "ultra-rares" (moins de 1 %). Par exemple, dans les données du pancréas humain de Baron, certaines cellules immunitaires ou cellules de Schwann ne représentent qu'une fraction de pourcent du total.

L'utilisation d'annotations pré-existantes (la "vérité terrain") nous permet d'évaluer objectivement si les algorithmes parviennent à retrouver ces populations ou s'ils les noient dans la masse. Le tableau 5 (disponible en Annexe B) résume l'ensemble des jeux de données de référence retenus pour notre étude, détaillant pour chacun d'eux l'espèce d'origine, le nombre de cellules et de gènes, le nombre de lots (batches) ainsi que la quantité de populations cellulaires rares.

Les données de séquençage d'ARN sur cellule unique (scRNA-seq) manipulées sont généralement structurées sous forme de matrices d'expression accompagnées de métadonnées. Par convention (notamment dans le format `AnnData` utilisé par notre pipeline), ces informations sont organisées de la façon suivante :

- **La matrice d'expression (X) :** De dimension $N \times M$ (où N est le nombre de cellules et M le nombre de gènes). Chaque entrée représente le niveau d'expression (par exemple le nombre de comptages d'U.M.I., pour *Unique Molecular Identifiers*) d'un gène spécifique dans une cellule donnée. Cette matrice est généralement très creuse (majoritairement remplie de zéros).
- **Les métadonnées cellulaires (*observations* ou *obs*) :** Une table de dimension $N \times D$ alignée sur les lignes de la matrice d'expression. Elle attribue à chaque cellule des informations de contexte comme le lot expérimental d'origine (*batch*) ou l'annotation biologique (*cell type*).
- **Les métadonnées des gènes (*variables* ou *var*) :** Une table de dimension $M \times G$ alignée sur les colonnes de la matrice d'expression, décrivant les caractéristiques des gènes (nom du gène, niveau de variabilité globale, etc.).

Le tableau 1 illustre un exemple conceptuel de cet alignement entre la matrice d'expression et les métadonnées cellulaires.

Identifiant Cellule	Matrice d'expression (X)			Métadonnées des cellules (obs)	
	Gène A	Gène B	Gène C	Lot (Batch)	Type cellulaire
Cellule_1	0	14	0	Lot_A	Cellule Beta
Cellule_2	3	0	1	Lot_A	Cellule Alpha
Cellule_3	0	0	0	Lot_B	Cellule de Schwann
Cellule_4	24	1	0	Lot_B	Cellule Beta
Cellule_5	0	0	8	Lot_C	Lymphocyte T

TABLE 1 – Structure conceptuelle d'un jeu de données scRNA-seq. La matrice d'expression à gauche est alignée ligne à ligne (par cellule) avec la table des métadonnées cellulaires à droite.

2.2 Traitements de données

Les données brutes issues du séquençage scRNA-seq sont extrêmement bruitées et pleines de zéros (des gènes non détectés). Il est donc indispensable de les nettoyer et de les préparer.

2.2.1 Pré-process (Le nettoyage initial)

Le traitement standard [6] consiste d'abord à filtrer les cellules de mauvaises qualités et les gènes très peu exprimés. Ensuite, on normalise les données pour que toutes les cellules soient comparables, puis on applique une transformation mathématique (le logarithme) pour atténuer les écarts extrêmes. Enfin, on ne conserve généralement que les gènes dits "hautement variables" (HVG), c'est-à-dire ceux qui différencient le mieux les cellules entre elles. Bien que ces étapes soient standardisées, elles présentent un risque pour les cellules rares : un filtrage trop agressif peut effacer les quelques gènes qui caractérisaient spécifiquement une toute petite population.

2.2.2 Batch effect (L'effet de lot)

L'effet de batch est un biais technique. Si l'on séquence le même tissu dans deux laboratoires différents, ou sur deux machines différentes, les cellules auront tendance à se regrouper par "laboratoire" plutôt que par "type cellulaire". Il existe des méthodes dédiées pour corriger ce problème (présentées dans le tableau 2), comme ComBat [10], Harmony [11], Scanorama [12], ou scVI [9]. L'objectif est d'aligner les différents lots pour effacer la signature technique. Cependant, la difficulté est de ne pas "sur-corriger" : en voulant effacer les différences techniques, on risque de gommer de vraies différences biologiques, en particulier celles des populations rares qui pourraient n'apparaître que dans un seul lot.

Une stratégie plus récente, utilisée ensuite dans notre modèle, consiste à apprendre une représentation qui conserve l'information utile tout en rendant le batch difficile à reconnaître. C'est l'idée des DANN (*Domain-Adversarial Neural Networks*) : un réseau principal apprend l'espace latent, tandis qu'un second réseau essaie de prédire le domaine ou le lot expérimental à partir de cet espace. Grâce à une couche d'inversion de gradient, l'encodeur est pénalisé lorsque le batch reste trop facile à prédire, ce qui l'encourage à produire une représentation

moins dépendante du lot [13]. Cette logique a déjà été appliquée à des données biologiques, notamment avec scDGN pour aligner des données scRNA-seq entre expériences [14], puis avec D2AN pour associer correction de batch et annotation des types cellulaires [15]. Des approches proches, comme ABC, ajoutent également un discriminateur de batch à un autoencodeur afin de limiter l'information technique dans l'espace latent [16]. Le point de vigilance reste le même que pour les autres méthodes : si le batch est fortement corrélé à une différence biologique réelle, la correction adversariale peut aussi affaiblir ce signal.

Outil	Méthodologie	Principe de correction
ComBat [10]	Modèle bayésien empirique	Ajustement des moyennes et variances des gènes par lot
DANN [13]	Réseau de neurones adversarial	Inversion de gradient pour masquer l'information de lot dans l'espace latent
Harmony [11]	Soft K-Means itératif	Alignement des cellules similaires de lots différents en PCA
Scanorama [12]	Mutual Nearest Neighbors (MNN)	Identification des cellules "panoramas" communes entre les lots
scVI [9]	Autoencodeur Variationnel (VAE)	Modélisation probabiliste du bruit et des batchs dans l'espace latent

TABLE 2 – Principaux outils de correction de l'effet de lot (Batch Effect).

2.3 Algorithmes de clustering

Une fois les données prêtes, l'objectif est de regrouper les cellules similaires.

2.3.1 Approches classiques

L'approche la plus courante en bioinformatique consiste à simplifier les données (généralement via une méthode appelée PCA), puis à regrouper directement les cellules avec un algorithme comme K-Means [17], ou bien à construire un graphe où chaque cellule est reliée à ses voisines les plus proches pour y identifier des communautés avec des algorithmes comme Louvain [18] ou Leiden [19]. Ces méthodes sont très rapides, faciles à utiliser et intégrées dans des outils standards comme Scanpy [7]. Toutefois, elles sont très sensibles au nettoyage initial des données. De plus, si une population est très petite, elle risque de ne pas former sa propre communauté dans le graphe et d'être absorbée par une population voisine plus grande.

2.3.2 Approches Deep Learning (Apprentissage Profond)

Pour aller plus loin, les méthodes basées sur le Deep Learning (l'intelligence artificielle) apprennent à représenter les cellules dans un nouvel espace mathématique (l'espace latent) de manière beaucoup plus complexe et performante que la PCA. Ces méthodes, souvent basées

sur des autoencodeurs, s'entraînent en essayant de minimiser une "fonction de perte" (ou *loss*). Cette *loss* est une information mathématique qui évalue la qualité de l'apprentissage et pénalise le modèle lorsqu'il commet des erreurs.

Pour bien comprendre, regardons sur quoi se base la perte totale (*total loss*) de différents modèles de l'état de l'art :

- **La perte de reconstruction** : Des modèles comme scVI [9] ou scMAE [20] cherchent avant tout à bien reconstruire le profil d'expression initial de la cellule. Si le modèle compresse mal l'information et n'arrive pas à deviner les gènes de départ, sa perte augmente.
- **La perte de clustering** : Des méthodes comme DESC [21], scDeepCluster [22] ou scNAME [23] ajoutent une contrainte supplémentaire. Elles mesurent les distances entre les cellules et forcent le modèle à regrouper les cellules qui se ressemblent de plus en plus au fil du temps (souvent en minimisant une mesure de différence appelée divergence KL).

Le cœur du problème avec les populations rares vient précisément de la manière dont cette perte totale est calculée. Le modèle cherche à minimiser cette perte globale sur l'ensemble des données. Les populations majoritaires (qui contiennent des milliers de cellules) vont avoir un poids énorme dans ce calcul. À l'inverse, si le modèle se trompe sur un type cellulaire rare (qui ne contient que quelques dizaines de cellules), cela fera à peine bouger la perte globale. Le réseau de neurones va donc avoir tendance à "ignorer" la précision sur les cellules rares pour optimiser sa représentation mathématique globale.

Pour résumer, le tableau 3 présente une synthèse des méthodes citées, des approches classiques jusqu'aux architectures Deep Learning et aux méthodes dédiées aux populations rares.

Famille	Méthode	Framework	Objectif / loss	Réf.	Source
Approches classiques	PCA	Projection linéaire	Maximisation de la variance expliquée	[24]	Philosophical Magazine
	K-Means	Centroïdes	Minimisation de l'inertie intra-cluster	[17]	Berkeley Symposium
	Louvain	Graphe de voisinage	Maximisation de la modularité	[18]	Journal of Statistical Mechanics
	Leiden	Graphe de voisinage	Modularité avec communautés mieux connectées	[19]	Scientific Reports
Deep Learning	scVI	VAE probabiliste	Reconstruction NB/ZINB + divergence KL	[9]	Nature Methods
	scMAE	Masked autoencoder	Reconstruction de gènes masqués	[20]	Bioinformatics
	DESC	Autoencodeur + clustering	Reconstruction + clustering par divergence KL	[21]	Nature Communications
	scDeepCluster	Autoencodeur ZINB	Reconstruction ZINB + clustering par divergence KL	[22]	Nature Machine Intelligence
	scNAME	Contrastive learning	Reconstruction + clustering + contraste de voisinage	[23]	Bioinformatics
Graphes profonds	scCDCG	AE + graph embedding	Reconstruction + transport optimal + graph cut	[25]	DASFAA
Cellules rares	GiniClust	Indice de Gini	Détection de gènes marqueurs atypiques	[26]	Genome Biology
	CellSIUS	Signatures locales	Identification de sous-populations minoritaires	[27]	Genome Biology
	scAIDE	Autoencodeur	Débruitage + clustering orienté populations rares	[28]	NAR Genomics and Bioinformatics
	DeepScena	Self-supervised	Raffinement coarse-to-fine des clusters rares	[29]	Briefings in Bioinformatics
	scCAD	Détection d'anomalies	Décomposition de clusters pour isoler les types rares	[30]	Nature Communications

TABLE 3 – Synthèse des méthodes de clustering et de détection de populations rares citées dans l'état de l'art. AE : autoencodeur ; VAE : autoencodeur variationnel ; KL : divergence de Kullback-Leibler ; NB/ZINB : lois binomiale négative et binomiale négative à inflation de zéros.

2.3.3 Les types cellulaires dans les algorithmes

Face à ces limites, l'identification des populations cellulaires minoritaires est devenue un enjeu central dans l'analyse des données single-cell RNA-seq. Plusieurs algorithmes ont ainsi été développés pour mieux détecter ces sous-populations rares. Tous s'appuient, d'une manière ou d'une autre, sur les profils d'expression des gènes, mais ils ne les exploitent pas de la même façon.

Certaines méthodes cherchent directement des gènes caractéristiques des cellules rares. Par exemple, **GiniClust** [26] utilise l'indice de Gini pour repérer des gènes exprimés dans seulement un petit nombre de cellules. Ces gènes peuvent alors signaler l'existence d'une

population rare qui serait difficile à voir avec un clustering classique. Dans une logique proche, **CellSIUS** [27] part de groupes cellulaires déjà formés, puis recherche à l'intérieur de ces groupes des signatures d'expression particulières permettant d'identifier des sous-populations minoritaires.

D'autres méthodes cherchent plutôt à améliorer la représentation globale des cellules avant de les regrouper. C'est le cas de **scAIDE** [28], qui utilise un autoencodeur pour réduire le bruit des données et produire une représentation plus claire des cellules. Une fois cette représentation obtenue, scAIDE applique une méthode de clustering conçue pour limiter le biais en faveur des grandes populations cellulaires. L'objectif est donc de mieux séparer les petits groupes de cellules, y compris lorsqu'ils sont très peu représentés dans le jeu de données.

Dans une approche proche, mais davantage hiérarchique, **DeepScena** [29] commence par identifier de grands groupes cellulaires, puis réanalyse progressivement certains de ces groupes afin de faire apparaître des sous-populations plus fines. À chaque niveau, les gènes les plus informatifs peuvent être recalculés dans le sous-groupe étudié, ce qui permet de mieux détecter des différences qui étaient invisibles à l'échelle globale. DeepScena ne donne donc pas directement plus de poids aux cellules rares dans sa fonction de perte ; il les révèle plutôt grâce à un raffinement progressif du clustering et à une meilleure modélisation des données single-cell.

Enfin, des méthodes plus récentes comme **scCAD** [30] abordent la question sous l'angle de la détection d'anomalies. L'idée est d'identifier les cellules dont le profil d'expression s'éloigne de celui des populations majoritaires, puis de décomposer progressivement les clusters afin d'isoler ces groupes atypiques.

Ces méthodes reposent donc sur une idée commune : un clustering global classique tend à favoriser les grandes populations cellulaires et peut masquer les cellules rares ou les états intermédiaires. Notre démarche, **scRAW**, s'inscrit dans cette continuité tout en proposant une stratégie différente. Plutôt que d'identifier les populations rares uniquement après la construction des clusters ou par une étape de raffinement, nous cherchons à intégrer cette contrainte directement pendant l'apprentissage du modèle. Concrètement, scRAW attribue un poids plus important aux cellules isolées ou faiblement représentées dans le calcul de la *loss*, afin qu'elles influencent davantage la construction de l'espace latent. L'objectif est ainsi de produire une représentation latente plus informative, dans laquelle les populations rares ne sont pas absorbées par les grands groupes cellulaires et étant mise en évidence tout au long de l'entraînement du modèle.

3 Comparaison de l'existant

Nos travaux ont débuté par une comparaison des méthodes de l'état de l'art traitant des données de séquençage d'ARN à cellule unique (scRNA-seq). À ce stade, nous souhaitions étudier des approches généralistes, qui ne traitaient ni de l'effet de lot (*batch effect*) ni du déséquilibre des populations cellulaires. Pour ce faire, nous nous sommes en partie appuyés sur le protocole et les sélections du benchmark scCluBench [31] où nous avons sélectionné les méthodes de deep learning basés sur des autoencodeurs et sur des graphs les plus performantes sur les métriques comparées. La problématique ensuite rencontrée pour comparer les méthodes

sélectionner était la façon dont on voulait effectuer les tests. Nous souhaitions à la fois pouvoir modifier à notre guise les hyperparamètres des méthodes, tout en ajustant les étapes de preprocessing. De plus la quantité des tests différents et des résultats disponibles m'a poussé à développer une solution logiciel le plus ergonomique possible, SCRBenchmark, afin d'automatiser, simplifier la mise en place des différentes expériences et l'exploration des résultats.

3.1 Logiciel

SCRBenchmark a été développé pour rendre les expériences de comparaison plus simples à construire, plus faciles à reproduire et plus lisibles pour des utilisateurs qui ne travaillent pas nécessairement dans le code au quotidien. Le logiciel regroupe dans un même environnement le chargement des données, le choix du protocole expérimental, le prétraitement, la sélection des méthodes, l'exécution des analyses et l'exploration des résultats. Sa structure logique est résumée dans la figure 1.

Le premier choix important a été de séparer l'interface utilisée pour configurer les expériences du moteur chargé d'exécuter les calculs. L'interface web guide l'utilisateur étape par étape, tandis que la ligne de commande permet de relancer la même analyse sur un serveur ou dans un environnement de calcul plus adapté aux expériences longues. Les neuf écrans principaux de l'interface, consultables en Annexe F (Figure 9), couvrent l'ensemble du parcours : importation des données, visualisation préliminaire, choix du type d'expérience, prétraitement, sélection des algorithmes, génération de la commande d'exécution, suivi du calcul, consultation des métriques et visualisation des UMAP finales.

SCRBenchmark propose ensuite deux modes d'analyse. Le mode standard utilise toutes les cellules ensemble et sert à observer directement le comportement d'une méthode sur un jeu de données. Le mode benchmark sépare au contraire les cellules en ensembles d'entraînement, de validation et de test, afin d'évaluer les méthodes dans des conditions plus contrôlées, notamment lorsqu'il s'agit de tester leur comportement sur des cellules ou des lots non vus pendant l'entraînement.

L'intérêt principal du logiciel est d'imposer un cadre commun à des méthodes très différentes. Le même pipeline de prétraitement peut être appliqué à toutes les méthodes : filtrage, normalisation, sélection des gènes les plus informatifs et, si nécessaire, correction de l'effet de lot. De la même manière, le moteur d'exécution attend de chaque algorithme les mêmes types de sorties : groupes de cellules prédits, représentation numérique des cellules lorsque la méthode en produit une, paramètres utilisés et temps d'exécution. Ce processus commun permet de s'assurer que les résultats seront issus du fonctionnement des algorithmes et non des données en entrée.

Enfin, SCRBenchmark sauvegarde les éléments nécessaires à l'analyse et à la reproductibilité : scores, labels prédits, embeddings, figures, paramètres et logs. Les visualisations générées automatiquement, comme les UMAP, les matrices de confusion ou les figures comparant entraînement et test, permettent d'explorer rapidement les résultats sans réécrire du code pour chaque expérience. L'outil a donc servi à la fois à produire les expériences et à en analyser les sorties de manière cohérente.

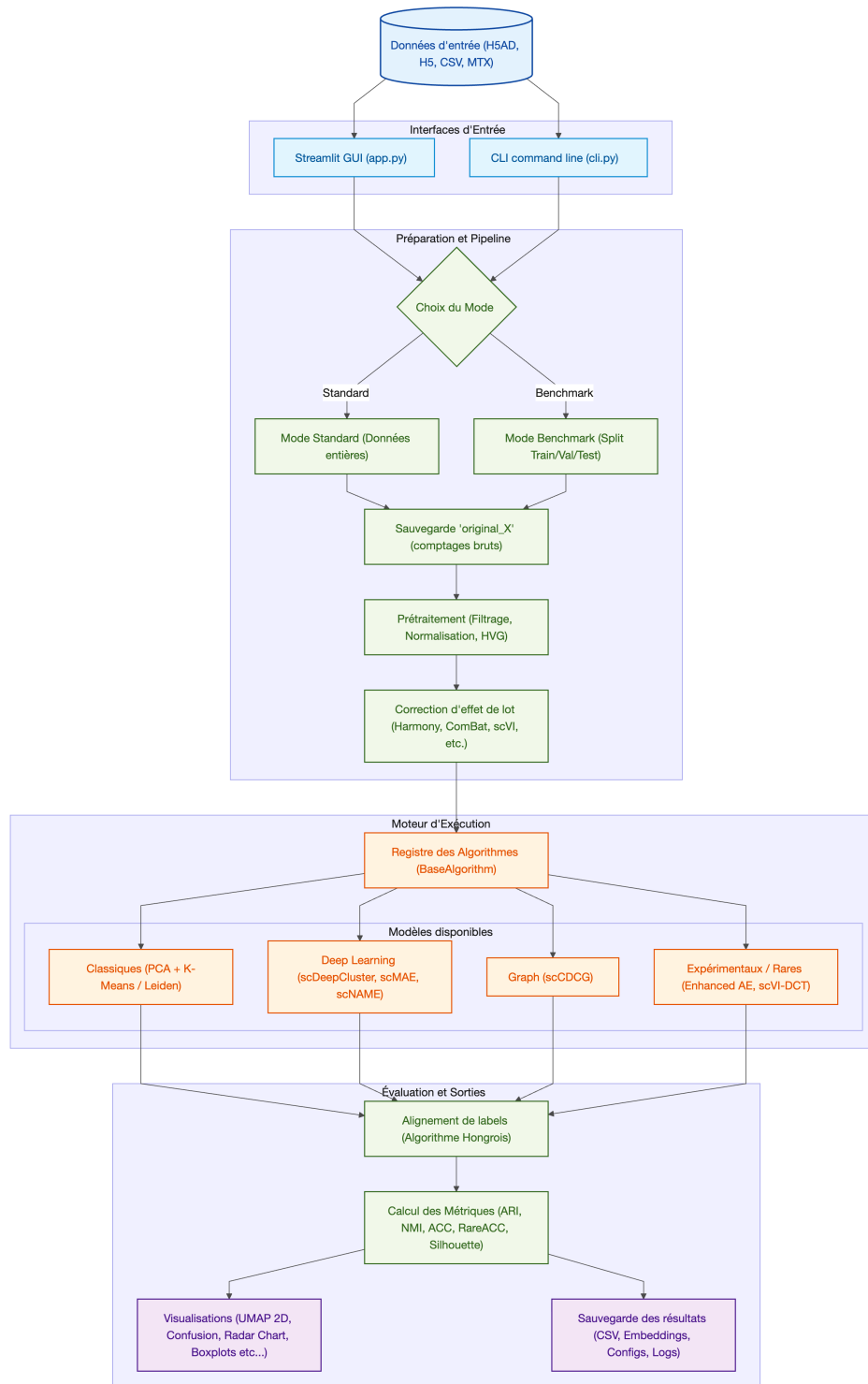


FIGURE 1 – Structure logique et flux de travail de la plateforme de benchmarking SCRBenchmark.

3.2 Expériences

3.2.1 Données et splits

Nous avons retenu le jeu de données Baron [32] pour les expériences présentées ici. Ce choix permettait de réunir dans un même cadre les deux difficultés au centre du mémoire : un effet de lot lié aux donneurs et un fort déséquilibre entre types cellulaires. Le jeu de données regroupe quatre donneurs humains, notés *human1* à *human4*, et constitue un standard de référence largement utilisé pour évaluer les méthodes de clustering en scRNA-seq. Les publications originales de plusieurs méthodes d'apprentissage profond comme scDeepCluster [22], scNAME [23], scMAE [20] et scCDCG [25], ainsi que des approches dédiées aux populations rares telles que CellSIUS [27], scAIDE [28], DeepScena [29], scCAD [30] et le benchmark scCluBench [31], s'appuient sur ce type de données de référence pour valider leurs performances.

Après filtrage, les expériences sur Baron portent sur 8 569 cellules réparties en quatorze types cellulaires. Le tableau 4 montre que les cellules *alpha* et *beta* représentent à elles seules plus de la moitié du jeu de données, tandis que plusieurs populations d'intérêt biologique, comme les *macrophage*, *mast*, *schwann* ou *t_cell*, représentent moins de 1 % des cellules. Ce jeu de données est donc particulièrement adapté pour tester si un algorithme conserve les petites populations au lieu d'optimiser uniquement les grandes classes.

Type cellulaire	Cellules	Proportion	Statut
<i>beta</i>	2 525	29,47 %	Fréquente
<i>alpha</i>	2 326	27,14 %	Fréquente
<i>ductal</i>	1 077	12,57 %	Fréquente
<i>acinar</i>	958	11,18 %	Fréquente
<i>delta</i>	601	7,01 %	Fréquente
<i>activated_stellate</i>	284	3,31 %	Rare
<i>gamma</i>	255	2,98 %	Rare
<i>endothelial</i>	252	2,94 %	Rare
<i>quiescent_stellate</i>	173	2,02 %	Rare
<i>macrophage</i>	55	0,64 %	Ultra-rare
<i>mast</i>	25	0,29 %	Ultra-rare
<i>epsilon</i>	18	0,21 %	Ultra-rare
<i>schwann</i>	13	0,15 %	Ultra-rare
<i>t_cell</i>	7	0,08 %	Ultra-rare

TABLE 4 – Composition du jeu de données Baron après filtrage. Les classes rares correspondent aux types représentant moins de 5 % des cellules, et les classes ultra-rares à ceux représentant moins de 1 %.

Différentes expériences ont été menées afin de répondre à une question simple : est-ce qu'un modèle qui apprend une représentation sur certaines cellules est ensuite capable de rester performant sur des cellules qu'il n'a jamais vues ? Cette question distingue deux cadres d'évaluation. Dans un cadre transductif, toutes les cellules sont disponibles au moment de l'apprentissage et le modèle construit ses groupes à partir de l'ensemble complet. Ce cadre est utile pour explorer un jeu de données déjà acquis, mais il ne dit pas vraiment ce qui se

passerait si l'on ajoutait de nouvelles cellules ensuite. Dans un cadre inductif, au contraire, le modèle est entraîné sur une partie des cellules, puis il doit projeter ou classer de nouvelles cellules sans être réentraîné depuis le début.

Pour rendre cette évaluation possible avec des modèles basés sur des autoencodeurs, nous avons utilisé leur encodeur comme outil de projection. Cette adaptation s'appuie sur une idée déjà présente dans les méthodes de deep clustering comme DEC [33] et IDEC [34], où les cellules sont projetées dans un espace latent avant d'être rapprochées de centres de clusters. L'objectif d'appliquer un modèle entraîné à de nouvelles cellules se rapproche quant à lui davantage de la logique de projection de requêtes sur une référence (*query-to-reference mapping*) proposée par scArches [35] en génomique sur cellule unique. Concrètement, une fois le modèle entraîné, ses poids sont gelés : cela signifie qu'ils ne sont plus modifiés. Les cellules d'entraînement sont projetées dans l'espace latent, puis les centres des clusters appris sont calculés dans cet espace. Les nouvelles cellules passent ensuite par le même prétraitement et par le même encodeur ; elles sont finalement rattachées au cluster dont le centre est le plus proche, ou à un groupe de référence construit pendant l'entraînement. Cette logique ne peut pas être appliquée de manière automatique à toutes les méthodes, car certaines reconstruisent un graphe ou un clustering global à chaque analyse et restent donc principalement transductives. Dans le panel présenté plus bas, les résultats inductifs correspondent donc aux méthodes pour lesquelles les sorties H123→H4 étaient disponibles, tandis que les six méthodes sont comparées dans le cadre transductif complet.

Sur Baron, nous avons conservé deux cadres principaux pour l'analyse des méthodes existantes. Le premier est transductif : l'ensemble des 8 569 cellules filtrées est analysé simultanément, puis les performances sont moyennées sur cinq graines aléatoires. Le second est inductif : les modèles sont entraînés sur les donneurs *human1*, *human2* et *human3*, puis évalués sur le donneur *human4*. Ce split H123→H4 permet de tester la capacité à projeter un donneur jamais vu pendant l'entraînement. Le principe est résumé dans la figure 2.

Situation représentée	Principe expérimental
Données initiales	Quatre donneurs sont disponibles avant filtrage : <i>human1</i> , <i>human2</i> , <i>human3</i> et <i>human4</i> . Leurs tailles et leurs compositions cellulaires diffèrent, ce qui crée un effet de lot exploitable pour tester la robustesse des méthodes.
Mode transductif	Toutes les cellules filtrées sont utilisées ensemble. Ce cadre mesure la capacité d’une méthode à retrouver la structure globale du jeu de données complet.
Split inductif	Les modèles sont entraînés sur <i>human1–human3</i> , puis évalués sur <i>human4</i> . Le test contient 1 303 cellules, dont plusieurs classes ultra-rares représentées par une seule cellule.
Lecture des résultats	Les métriques globales sont comparées aux erreurs par type cellulaire afin de vérifier si les bonnes performances moyennes masquent des confusions sur les petites populations.

Note : Le mode inductif présenté dans les résultats correspond au split $H123 \rightarrow H4$.

FIGURE 2 – Représentation simplifiée des principaux cadres expérimentaux utilisés sur le jeu de données Baron.

Ces deux cadres permettent de mesurer des propriétés complémentaires. Le mode transductif indique si une méthode retrouve correctement la structure globale du jeu de données complet, mais il peut être dominé par les grands types cellulaires et les lots majoritaires. Le mode inductif teste au contraire la capacité à projeter des cellules non vues, dans un contexte où le test contient moins de cellules et où les populations rares deviennent parfois représentées par quelques exemples seulement. Dans chaque cas, les performances ont été évaluées avec des métriques globales, mais aussi avec des métriques centrées sur les petites populations. Nous avons défini les classes rares comme les types cellulaires représentant moins de 5 % des cellules du jeu Baron complet, et les classes ultra-rares comme celles représentant moins de 1 %.

3.2.2 Méthodes

Nous avons alors lancé l’analyse sur un ensemble restreint de méthodes représentatives de l’état de l’art à partir des meilleurs algorithmes de scCluBench [31] : PCA suivie de clustering K-Means ou de Leiden, scDeepCluster, scMAE, scNAME et scCDCG. Les méthodes ont été exécutées avec les recommandations des auteurs lorsque celles-ci étaient disponibles, en conservant les hyperparamètres standards afin d’éviter d’optimiser une méthode au détriment des autres.

Le prétraitement a été identique pour toutes les méthodes afin que les différences observées proviennent principalement des algorithmes et non de la préparation des données. Les cellules et les gènes très peu informatifs ont d’abord été filtrés, puis les comptes ont été normalisés à 10 000 par cellule et transformés par logarithme. Nous avons ensuite conservé les 2 000 gènes hautement variables (*highly variable genes*, HVG) selon la stratégie de Seurat, avant une mise à l’échelle des données. Ce nombre de gènes constitue un compromis standard : il réduit le

bruit et le coût de calcul tout en conservant suffisamment d'information pour comparer les méthodes sur Baron.

Dans les expériences avec séparation entraînement/validation/test, la sélection des gènes variables a été réalisée à partir des données d'entraînement afin d'éviter d'utiliser indirectement de l'information provenant du test. Les expériences transductives sur le jeu complet ont été répétées cinq fois avec des graines aléatoires différentes. Cette répétition était nécessaire car plusieurs méthodes d'apprentissage profond dépendent de l'initialisation du modèle, et parce que les résultats d'un seul lancement peuvent donner une vision trop optimiste ou trop pessimiste des performances réelles. Le cadre inductif présenté dans le panel reprend les sorties disponibles pour le split H123→H4.

Afin de comparer les méthodes de manière homogène, les mêmes métriques ont été calculées pour toutes les expériences. La difficulté principale vient du fait qu'un algorithme de clustering ne prédit pas directement des noms de types cellulaires. Il produit seulement des groupes numérotés, par exemple cluster 0, cluster 1 ou cluster 2. Ces numéros sont arbitraires : le cluster 0 peut correspondre aux cellules alpha dans un lancement, mais à un autre type cellulaire dans un autre lancement. Avant de calculer les métriques de type accuracy, il faut donc associer les clusters prédits aux annotations biologiques de référence.

Pour réaliser cette association, nous avons construit une matrice de correspondance entre les clusters prédits et les classes biologiques connues, puis nous avons appliqué l'algorithme hongrois. Cet algorithme résout un problème d'appariement optimal : il cherche la correspondance cluster-classe qui maximise le nombre total de cellules correctement attribuées. Cette stratégie est utilisée dans des benchmarks de clustering comme scCluBench pour l'annotation par meilleur appariement [31], et correspond à la méthode hongroise proposée initialement pour les problèmes d'affectation [36]. Après cette étape, les métriques ACC, BalancedACC, RareACC et UltraRareACC peuvent être interprétées comme des performances de classification sur les types cellulaires. À l'inverse, l'ARI et le NMI comparent directement deux clusters et ne nécessitent pas ce renommage explicite de ces derniers. L'ensemble des métriques d'évaluation utilisées, avec leurs formules et leurs rôles respectifs, est récapitulé dans le tableau 6 (disponible en Annexe C).

Dans les formules suivantes, y_i désigne l'annotation biologique de la cellule i , $\hat{y}_i$ le cluster prédit par la méthode, n le nombre total de cellules, $\mathcal{C}$ l'ensemble des classes biologiques et π l'appariement optimal obtenu par l'algorithme hongrois.

Les métriques globales comme l'ARI, le NMI ou l'ACC donnent une vision générale de la qualité du regroupement, mais elles peuvent rester élevées même si une petite population est mal retrouvée. Les métriques BalancedACC, RareACC et UltraRareACC complètent donc cette lecture en donnant davantage de poids à chaque type cellulaire, indépendamment de sa taille. Elles sont particulièrement importantes dans notre cadre, car l'objectif n'est pas seulement de retrouver les grandes populations majoritaires, mais aussi de vérifier que les cellules rares ne sont pas absorbées dans des groupes dominants.

3.2.3 Résultats

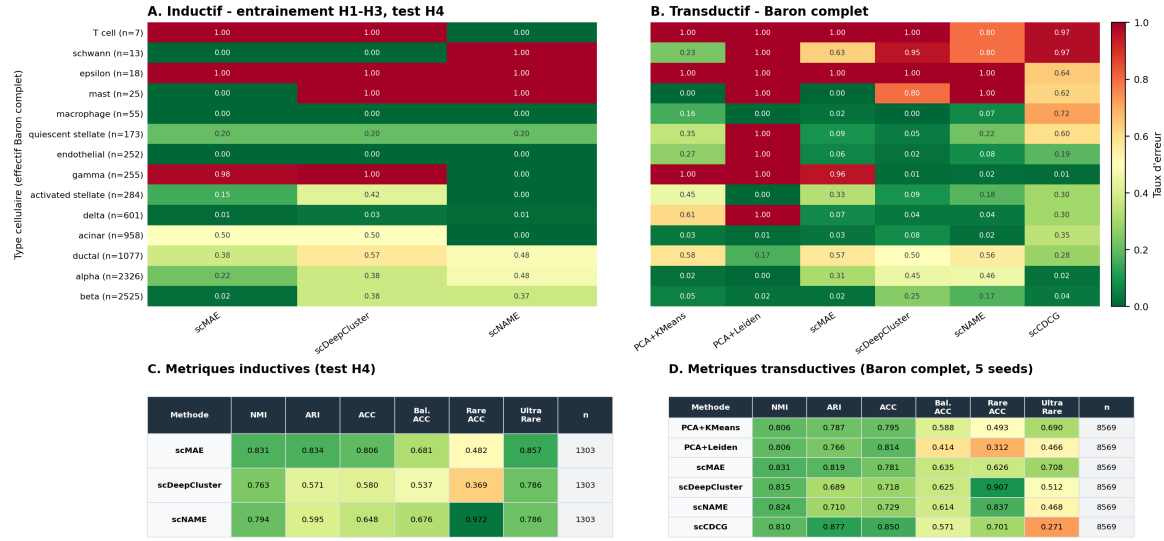
Les résultats obtenus sur Baron montrent que les métriques globales seules ne suffisent pas à caractériser la qualité biologique du clustering. La figure 3 compare deux cadres complémentaires : un cadre inductif, évalué sur le donneur *human4*, et un cadre transductif, évalué sur le jeu complet. Le panel présente le taux d'erreur par type cellulaire et par méthode ainsi que les métriques globales associées. Le taux d'erreur est défini comme $1 - \text{rappel}$ après appariement hongrois des clusters.

Dans le cadre transductif, plusieurs méthodes obtiennent des scores globaux élevés. scMAE atteint par exemple le meilleur NMI moyen (0,831), tandis que scCDCG obtient le meilleur ARI (0,877) et la meilleure ACC (0,850). Pourtant, les premières lignes de la heatmap restent très contrastées : les cellules *t_cell*, *schwann*, *epsilon* ou *mast* sont souvent mal retrouvées, alors que les grands types cellulaires comme *alpha*, *beta*, *ductal* ou *acinar* pèsent beaucoup plus fortement dans les scores globaux. Ce contraste explique pourquoi la BalancedACC, la RareACC et l'UltraRareACC sont nécessaires pour interpréter les résultats.

Le mode inductif paraît parfois plus favorable pour certaines populations minoritaires, par exemple avec une RareACC élevée pour scNAME sur le test H4. Cette lecture doit cependant rester prudente. L'évaluation inductive porte sur 1 303 cellules, contre 8 569 cellules en transductif, et plusieurs classes ultra-rares du test ne contiennent qu'une seule cellule. Si cette unique cellule est correctement attribuée, éventuellement par hasard, le taux d'erreur devient nul pour toute la ligne ; si elle est mal attribuée, il devient maximal. Cette hypothèse d'un effet de petit effectif est donc plausible et explique pourquoi certains résultats peuvent sembler meilleurs en inductif qu'en transductif sans prouver une meilleure généralisation. Elle justifie de toujours lire les scores globaux avec les effectifs par type cellulaire, la RareACC, l'UltraRareACC et la stabilité entre graines.

Baron - taux d'erreur par type cellulaire et scores globaux

Erreur = 1 - rappel apres appariement hongrois ; transductif moyenne sur 5 seeds, inductif sur le split H123 -> H4. Les lignes sont ordonnees des types les plus rares aux plus frequents.



Note : le test inductif ne contient que 1303 cellules ; les types ultra-rares peuvent n'y avoir qu'une cellule, ce qui rend leur taux d'erreur tres sensible au hasard.

FIGURE 3 – Panel de résultats sur le jeu de données Baron. A–B : taux d’erreur par type cellulaire et par méthode, en mode inductif sur le test H123→H4 et en mode transductif sur le jeu complet. C–D : métriques globales associées. Les résultats transductifs sont moyennés sur cinq graines ; le cadre inductif reprend les sorties disponibles pour le split H123→H4. Les types cellulaires sont ordonnés par effectif croissant dans le jeu complet. Les scores NMI/ARI peuvent rester corrects alors que l’identification des populations rares dépend fortement du nombre de cellules évaluées.

Ces observations motivent directement le développement de scRAW : l’objectif n’est pas seulement d’améliorer un score moyen, mais de construire une représentation dans laquelle les populations minoritaires conservent assez d’influence pour ne pas être noyées dans les classes dominantes.

4 scRAW

4.1 Positionnement et motivation

scRAW a été développé à partir du constat présenté dans la section précédente : les méthodes de clustering peuvent obtenir de bons scores globaux tout en retrouvant mal les populations cellulaires les plus petites. L’objectif n’est donc pas seulement de construire un nouvel autoencodeur, mais de modifier la manière dont les cellules contribuent à l’apprentissage. Les cellules situées dans de petits groupes ou dans des zones peu denses de l’espace latent reçoivent un poids plus élevé dans la fonction de perte. Elles influencent ainsi davantage la représentation apprise.

La méthode conserve une structure volontairement simple. Elle repose sur un autoencodeur MLP, une pondération dynamique des cellules, une contrainte locale de type triplet loss et une correction adversariale de l’effet de batch lorsque plusieurs lots sont présents. Dans la suite,

la description correspond uniquement à la configuration par défaut de scRAW. Les autres variantes de configuration ne sont pas détaillées ici.

4.2 Pipeline

Le pipeline général de scRAW est présenté dans la figure 4. Nous appliquons aux données brutes les étapes de prétraitement standard, identiques à celles détaillées dans la comparaison de l'existant. De plus, aucune correction de batch séparée n'est appliquée à cette étape ; la prise en compte du batch intervient ensuite directement au sein du modèle.

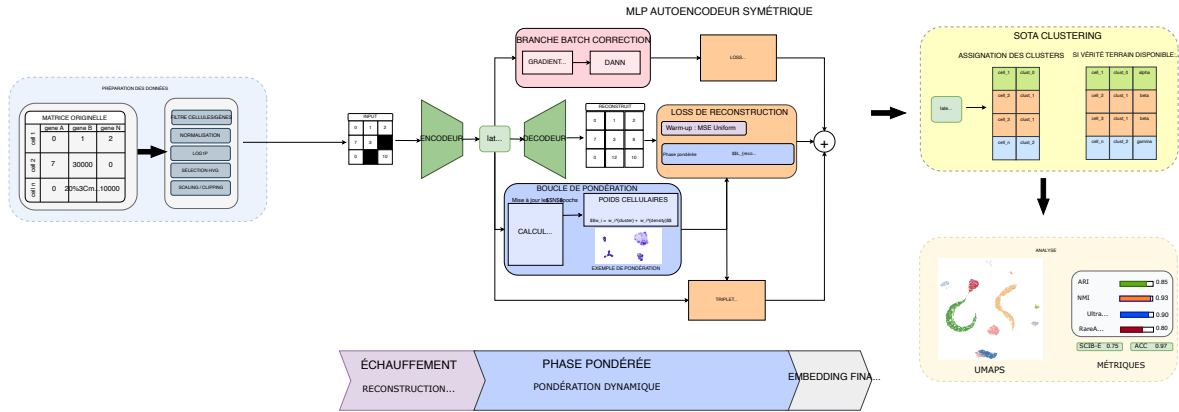


FIGURE 4 – Pipeline général de scRAW. Le modèle apprend un espace latent par autoencodeur, met à jour des poids cellulaires à partir de pseudo-clusters et de la densité locale, puis réalise le clustering final dans l'espace latent.

La matrice prétraitée est ensuite utilisée comme entrée d’un autoencodeur symétrique. Dans la configuration par défaut, l’encodeur comporte trois couches cachées de tailles 512, 256 et 128, puis projette chaque cellule dans un espace latent de dimension 256. Le décodeur suit la structure inverse afin de reconstruire le profil d’expression. Le modèle est entraîné pendant 120 époques, avec un *batch size* de 192, un dropout de 0,3 et un taux d’apprentissage de $1,64 \times 10^{-3}$. Les valeurs complètes sont données en Annexe D.

L’entraînement est divisé en deux phases. Pendant les 55 premières époques, toutes les cellules contribuent de manière uniforme à la reconstruction. Cette phase sert à stabiliser l’espace latent avant d’introduire la pondération. À partir de l’époque 55, scRAW recalcule périodiquement des pseudo-labels et des poids cellulaires à partir de l’espace latent courant. Dans la configuration par défaut, les pseudo-labels sont obtenus avec Leiden et les poids sont mis à jour toutes les 20 époques. Les poids déjà présents sont lissés par une moyenne mobile, avec un momentum de 0,688, afin d’éviter des changements trop brusques au cours de l’apprentissage.

À la fin de l’entraînement, l’encodeur produit l’embedding final de chaque cellule. Le clustering final est ensuite réalisé avec HDBSCAN dans cet espace latent, avec `min_cluster_size=8`, `min_samples=6` et la stratégie de sélection `eom`. Les points considérés comme bruit par HDBSCAN ne sont pas réassignés dans cette configuration. Les sorties principales de scRAW sont donc l’espace latent, les labels de clusters prédits, les poids cellulaires finaux, le modèle entraîné, l’historique des pertes et les métriques calculées par le pipeline d’évaluation.

4.3 Fonctions de perte et variantes

La perte principale de scRAW est une perte de reconstruction. Dans la configuration par défaut, il s’agit d’une erreur quadratique moyenne (MSE) calculée entre la matrice prétraitée X et sa reconstruction $\hat{X}$. Pour une cellule i , on peut l’écrire simplement :

$$\mathcal{L}_{\text{recon},i} = \frac{1}{G} \sum_{g=1}^G (\hat{X}_{i,g} - X_{i,g})^2,$$

où G est le nombre de gènes conservés après prétraitement. Une fraction de 10 % des entrées est masquée aléatoirement pendant l’entraînement, avec une valeur de masquage fixée à 0. Les positions masquées reçoivent un poids interne de 0,8 dans le calcul de la reconstruction. Ce masquage est aussi actif pendant la phase pondérée.

La différence principale avec un autoencodeur standard vient du facteur ω_i , qui module la contribution de chaque cellule :

$$\mathcal{L}_{\text{recon,ponderee}} = \frac{1}{N} \sum_{i=1}^N \omega_i \mathcal{L}_{\text{recon},i}.$$

Pendant la phase initiale, $\omega_i = 1$ pour toutes les cellules. Pendant la phase pondérée, ω_i est recalculé à partir de deux informations. La première est la taille du pseudo-cluster Leiden auquel appartient la cellule : plus le pseudo-cluster est petit, plus le poids augmente. La seconde est la densité locale dans l’espace latent, estimée par la distance au 15^e plus proche

voisin : une cellule plus isolée reçoit un poids plus élevé. Dans la configuration par défaut, ces deux composantes sont fusionnées de manière multiplicative :

$$\tilde{\omega}_i = \omega_i^{\text{cluster}} \times \omega_i^{\text{densite}}.$$

Le résultat est normalisé pour avoir une moyenne proche de 1, puis limité entre 0,385 et 10. Cette borne évite qu’une cellule domine entièrement la perte, tout en empêchant les cellules très fréquentes de disparaître de l’apprentissage.

La perte totale ajoute deux termes à cette reconstruction pondérée :

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{recon,ponderee}} + \rho(e)\lambda_{\text{triplet}}\mathcal{L}_{\text{triplet}} + \lambda_{\text{batch}}\mathcal{L}_{\text{batch}}.$$

Le facteur $\rho(e)$ augmente progressivement le poids effectif du terme triplet au début de son activation. Ce terme est activé à partir de l’époque 60, avec un poids $\lambda_{\text{triplet}} = 0,0501$. Il utilise comme ancres les cellules dont le poids est supérieur ou égal à 1,2, dans la limite de 64 ancres par mini-batch. Pour chaque ancre, la méthode cherche une cellule positive dans le même pseudo-cluster et une cellule négative dans un autre pseudo-cluster. Cette contrainte agit directement sur l’espace latent : elle encourage les cellules rares ou isolées à rester proches de cellules similaires, tout en s’éloignant des groupes voisins.

Le terme $\mathcal{L}_{\text{batch}}$ correspond à la correction adversariale de l’effet de lot. Une petite tête de classification essaie de prédire le batch à partir de l’embedding latent. Grâce à l’inversion de gradient, l’encodeur est poussé à produire une représentation dans laquelle le batch est moins reconnaissable. Dans la configuration par défaut, cette branche est activée si au moins deux batches sont disponibles, avec $\lambda_{\text{batch}} = 0,1176$. Le signal adversarial commence à l’époque 30 et augmente progressivement sur 30 époques.

La variante décrite ici correspond donc à la version complète de la configuration par défaut : reconstruction MSE, pondération dynamique par rareté et densité, triplet loss centrée sur les cellules de fort poids, correction adversariale de batch et clustering final HDBSCAN. Le code permet d’isoler certaines de ces composantes pour des ablations, mais ces variantes ne sont pas utilisées pour définir la configuration de référence décrite dans cette section.

4.4 Recherche d’hyperparamètres

La configuration par défaut n’a pas été fixée à partir d’un seul essai. Elle provient d’une suite de recherches d’hyperparamètres. L’objectif était de conserver une bonne performance globale tout en évitant que les populations rares soient absorbées par les classes majoritaires. Les paramètres explorés concernaient l’architecture de l’autoencodeur, la dimension latente, le nombre d’époques, le taux d’apprentissage, la reconstruction, la pondération dynamique, la triplet loss, la correction adversariale de batch et le clustering final.

La recherche a été conduite avec Optuna [37]. Dans ce cadre, chaque essai correspond à une configuration complète. Optuna propose d’abord des configurations initiales, puis utilise les résultats déjà observés pour privilégier les zones de l’espace de recherche qui semblent plus prometteuses. Pour scRAW, chaque essai est exécuté avec le même pipeline que les autres expériences : préparation de la configuration, entraînement du modèle, clustering final, calcul

des métriques et sauvegarde des résultats. Le score optimisé est un score composite construit à partir des métriques de clustering, afin de ne pas sélectionner uniquement une configuration bonne sur l'ARI ou la NMI mais faible sur les classes rares.

La première étape a été une recherche large sur le jeu de données Baron Human Pancreas. Cette recherche contient 450 essais Optuna, avec un échantillonneur TPE et 32 essais initiaux. Le score combinait ARI, NMI, F1 macro, BalancedACC et RareACC, avec les poids respectifs 0,30, 0,25, 0,20, 0,15 et 0,10. Cette phase a permis d'identifier une première configuration très performante, dont le score moyen Optuna est de 0,8978. Les dix meilleurs essais ont ensuite été relancés avec trois graines indépendantes pour limiter l'effet d'une initialisation favorable. Dans cette consolidation, cette même configuration reste le meilleur candidat, avec un score moyen de 0,8717 sur les trois répétitions. Elle constitue donc un bon point de départ, mais reste optimisée sur un seul jeu de données.

La deuxième étape a cherché une configuration plus générale. L'espace de recherche a été restreint à partir des meilleurs essais sur Baron, puis évalué sur huit jeux de données d'entraînement : Baron Human Pancreas, Kang PBMC, Human testis GSE112013, Macaque retina bipolar, Paul15 bone marrow, Tabula Muris liver, BBAG094 Zeisel et BBAG094 spleen. Cette recherche contient 80 configurations Optuna, chacune évaluée sur les huit datasets d'entraînement, ce qui correspond à 80 évaluations par dataset dans la phase principale. Le score par dataset utilisait ARI, NMI, ACC, F1 macro, BalancedACC, RareACC et UltraRareACC, avec les poids 0,20, 0,17, 0,15, 0,13, 0,10, 0,10 et 0,15. Le score agrégé était calculé comme 0,8 fois la moyenne des scores et 0,2 fois la moyenne du quart inférieur. Ce choix pénalise les configurations qui fonctionnent bien en moyenne mais échouent sur quelques datasets. Les meilleurs candidats ont ensuite été réévalués sur plusieurs graines et sur des datasets de validation.

La troisième étape a encore réduit l'espace de recherche autour des meilleurs comportements observés, en ne conservant que les familles de paramètres jugées stables : reconstruction MSE, transformation $\log_{1p}$, pseudo-labels Leiden, clustering final hybride, et une plage plus étroite pour les paramètres de pondération, de triplet loss, de DANN et de HDBSCAN. Elle comporte 19 essais, avec arrêt anticipé des candidats trop faibles. Le score utilisé combine le score moyen, l'écart moyen à la meilleure méthode de référence sur chaque dataset, le pire écart observé et le taux de datasets où notre méthode est au moins au niveau de cette référence.

Le meilleur candidat stable présente un score combiné de 0,6728 sur 13 datasets évalués. Cette configuration est ensuite devenue la configuration par défaut utilisée dans les expériences présentées ici. Les valeurs exactes sont données en Annexe D. Les figures d'importance des paramètres sont regroupées en Annexe E. Elles montrent que les paramètres les plus déterminants changent selon l'échelle de recherche : la stratégie de clustering final et HDBSCAN dominent la recherche Baron, tandis que la recherche multi-datasets fait ressortir le poids minimal des cellules, le nombre d'époques et la correction adversariale. Dans la recherche finale, les paramètres les plus importants sont surtout `adversarial_lambda`, `hdbscan_min_cluster_size`, `adversarial_start_epoch`, `lr` et `min_cell_weight`. Cela confirme que la configuration finale dépend à la fois de la représentation apprise, de la pondération des cellules rares et de la stratégie de clustering finale.

4.5 Résultats et discussion

- Comparer scRAW aux meilleures méthodes de l'état de l'art sur les datasets retenus.
- Montrer les gains et les limites : scores, UMAP, matrices de confusion, visualisation des poids cellulaires.
- Discuter le comportement sur classes rares, classes fréquentes et batchs difficiles.
- Présenter le coût de calcul et les difficultés rencontrées.
- Terminer par ce que scRAW valide et ce qui reste fragile avant les extensions.

5 Travaux complémentaires

Idée de la section

Regrouper les différentes expériences complémentaires : transfert de la loss et passage à un mode inductif.

5.0 Transfert de loss

- Présenter l'idée : transférer la loss de reconstruction pondérée ou la logique rare-aware vers d'autres algorithmes de deep learning.
- Expliquer pourquoi c'est intéressant : séparer la contribution méthodologique de scRAW de son architecture précise.
- Décrire le protocole expérimental : méthodes ciblées, variantes avec et sans loss transférée, datasets, métriques.
- Montrer les premiers résultats : gains, échecs, stabilité, coût.
- Discuter les limites : compatibilité des architectures, difficulté d'intégration, risque de sur-pondération des petites populations.

5.1 Induction

- Distinguer transductif et inductif : entraîner sur un ensemble de cellules puis prédire ou projeter de nouvelles cellules non vues.
- Décrire le protocole : train sur certains batchs, gel du prétraitement et du modèle, prédiction sur batchs non vus.
- Préciser les artefacts sauvegardés : modèle, état du prétraitement, gènes retenus, centroïdes de référence, configuration.
- Expliquer pourquoi l'inductif rapproche scRAW d'un usage réel : nouvelles cellules, nouveau batch, réanalyse rapide sans tout réentraîner.
- Présenter les résultats comparant inductif et transductif lorsque c'est possible.
- Discuter les limites : batch non représentatif, classes absentes du train, baisse possible des performances rares.

5.2 Interprétation biologique ?

- À définir

Conclusion finale à rédiger

- *Résumer les contributions : SCRBenchmark, comparaison standardisée, scRAW, optimisation et premières extensions.*
- *Revenir à la question directrice : préserver les populations rares sans sacrifier entièrement la performance globale.*
- *Présenter les limites : taille des datasets, vérité terrain, sensibilité aux hyperparamètres, coût de calcul, validation biologique.*
- *Ouvrir sur les perspectives : stabilisation de l'inductif, transfert de loss, validation par marqueurs, intégration dans un workflow utilisateur.*

Figures et tableaux restants à préparer

- *Tableau ou figures principal des performances :*
- *UMAP comparatifs :*
- *Matrices de confusion :*
- *Schéma du pipeline scRAW :*
- *Visualisation des poids cellulaires de scRAW :*
- *Tableau d'ablation : variantes de scRAW, pertes, poids, clustering final.*
- *Figure de recherche d'hyperparamètres : résumé Optuna ou graphiques montrant le compromis entre performance globale, performance rare et stabilité.*
- *Figure inductive :*
- *Tableau du coût de calcul :*
- *Figure ou tableau d'analyse biologique :*

A Structure et organigramme du LaBRI

Le Laboratoire Bordelais de Recherche en Informatique (LaBRI), UMR 5800, au sein duquel s'est déroulé ce stage de fin d'études au sein de l'équipe BKB, est structuré autour de plusieurs grands pôles :

- **Direction** : Pascal Desbarats (Directeur par intérim), Nicolas Hanusse (Directeur adjoint par intérim), Serge Chaumette (Chargé de mission transfert et valorisation).
- **Administration** (Administratrice d'unité : Christine Richard) :
 - *Équipe administrative* (resp. Cathy Roubineau) : Accueil (Claire Douvier, Laurence Pinosa), Assistante administrative (Safia Amsili), Assistante de communication (Auriane Dantès).
 - *Équipe financière* (resp. Chloé Godard) : Gestionnaires (Isabelle Garcia, Anthony Gau, Emmanuelle Lesage, Elia Meyre).
 - *Assistante du SCRIME* : Annick Mersier.
- **Recherche** (structurée en plusieurs départements) :
 - *Image et Son* (Fabien Baldacci)
 - *Combinatoire et Algorithmique* (Cyril Gavaille)
 - *Systèmes et Données* (Jean-Rémy Falleri), département auquel appartient l'équipe BKB.
 - *Méthodes et Modèles formels* (Laurent Bienvenu)
 - *Supports et AlgoriThmes pour les Applications Numériques hAutes performanceS* (Pierre Ramet)
- **Appui à la recherche** :
 - *Ingénieurs affectés sur projet* : Nathalie Furmento, Frédéric Lalanne, Boris Mansencal, Gérald Point.
 - *Équipe soutien "Systèmes - Réseaux"* (resp. Pascal Ung) : Ingénieurs ASR (Michaël Fontaine, Emmanuel Fontaine, Gabriel Larrouy), Technicien ASR (Alexandre Issouli), Ingénieur DEV-Web (Pierre Lacroix).
- **Autre** : Assistant de prévention (Gérald Point).

L'organigramme structurel simplifié correspondant est présenté dans la Figure 5.

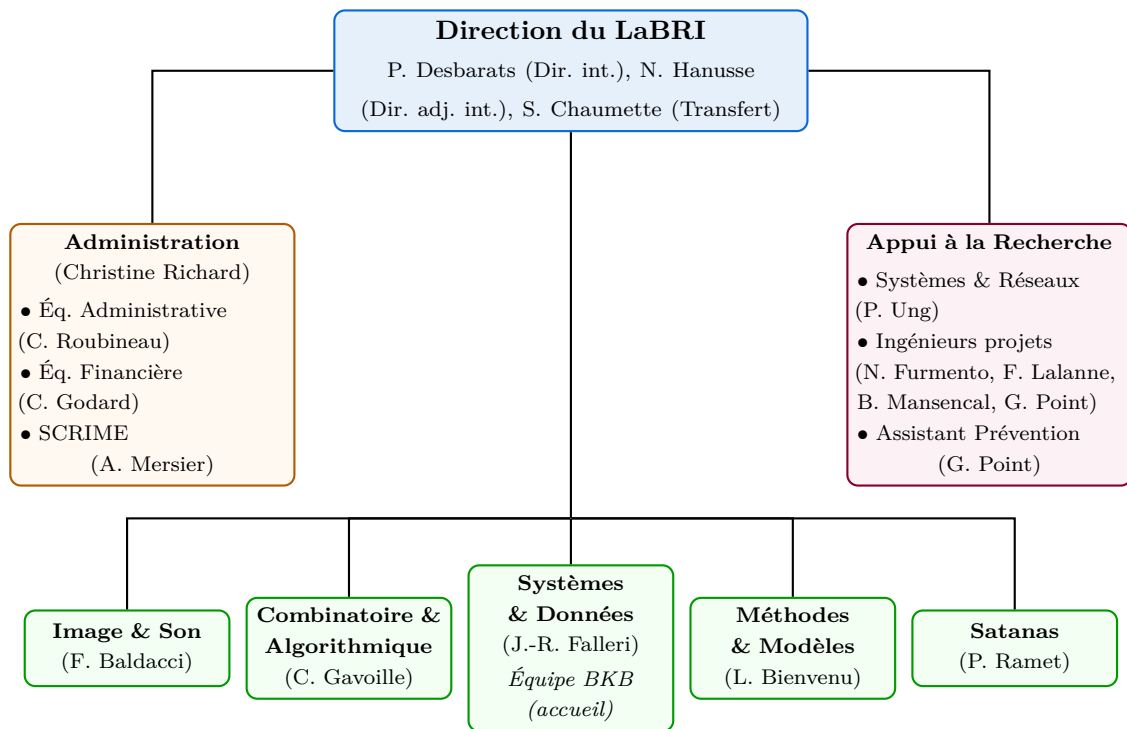


FIGURE 5 – Organigramme structurel simplifié du LaBRI.

B Détails des datasets de l'état de l'art

Cette section fournit les détails des différents jeux de données scRNA-seq de référence utilisés dans l'étude, y compris l'espèce, les tailles (cellules et gènes), le nombre de lots et la quantité de populations cellulaires rares.

Nom du Dataset	Espèce	Cellules	Gènes	Batches	Classes Rares ($< 5\%$)
BBAG094 Zeisel	Souris	3 006	19 972	10	4
BBAG094 spleen	Souris	9 552	23 341	2	3
Baron Human Pancreas	Humain	8 569	20 125	4	9
Human testis GSE112013	Humain	6 490	27 477	6	4
Kang PBMC	Humain	24 679	35 635	16	1
Macaque retina bipolar	Macaque	30 302	36 162	30	4
Human Pancreas	Humain	16 400	14 813	9	9
Pancreas w/o Baron	Humain	6 339	14 813	4	7
Paul15 bone marrow	Souris	2 730	3 451	1	9
SCIB Human Pancreas 1	Humain	1 937	20 125	1	8
SCIB Human Pancreas 2	Humain	1 724	20 125	1	10
SCIB Mouse Pancreas 1	Souris	822	14 878	1	9
Tabula Muris liver	Souris	2 859	22 966	1	5

TABLE 5 – Récapitulatif des jeux de données utilisés dans nos expériences.

C Tableau des métriques d'évaluation

Cette section regroupe les définitions, formules mathématiques et interprétations des métriques utilisées pour évaluer et comparer les performances des algorithmes de clustering, tant sur l'aspect global que sur la détection des classes rares et ultra-rares.

Métrique	Rôle	Formule ou principe	Interprétation
ARI	Clustering global	$ARI = \frac{RI - E[RI]}{\max(RI) - E[RI]}$, où RI mesure l'accord entre paires de cellules.	Mesure si deux cellules regroupées ensemble dans la vérité biologique le sont aussi dans la prédiction, en corrigeant l'accord attendu au hasard.
NMI	Clustering global	$NMI(Y, \hat{Y}) = \frac{2I(Y; \hat{Y})}{H(Y) + H(\hat{Y})}$	Mesure la quantité d'information partagée entre les annotations biologiques et les groupes prédits.
ACC	Classification après appariement des clusters	$ACC = \frac{1}{n} \max_{\pi} \sum_{i=1}^n \mathbf{1}\{y_i = \pi(\hat{y}_i)\}$	Donne la proportion globale de cellules correctement attribuées après correspondance entre clusters et classes.
BalancedACC	Classification équilibrée	$BalancedACC = \frac{1}{ C } \sum_{c \in C} \frac{TP_c}{N_c}$	Calcule la moyenne des rappels par classe, afin qu'une classe très abondante ne masque pas les erreurs sur les classes petites.
RareACC	Classification des classes rares	$RareACC = \frac{1}{ \mathcal{R} } \sum_{c \in \mathcal{R}} \frac{TP_c}{N_c}$, avec $\mathcal{R} = \{c \mid N_c/n < 5\%\}$	Évalue spécifiquement les types cellulaires rares, qui représentent moins de 5 % des cellules.
UltraRareACC	Classification des classes ultra-rares	$UltraRareACC = \frac{1}{ \mathcal{U} } \sum_{c \in \mathcal{U}} \frac{TP_c}{N_c}$, avec $\mathcal{U} = \{c \mid N_c/n < 1\%\}$	Mesure la capacité de la méthode à préserver les populations les plus minoritaires.
Temps	Coût computationnel	$t_{exec} = t_{fin} - t_{debut}$	Permet de comparer les méthodes non seulement sur leur qualité de clustering, mais aussi sur leur coût pratique.

TABLE 6 – Métriques utilisées pour comparer les méthodes. TP_c correspond au nombre de cellules de la classe c correctement attribuées après appariement, et N_c au nombre total de cellules de cette classe.

D Hyperparamètres de scRAW `stable_default`

Cette annexe détaille les hyperparamètres du preset `stable_default` utilisé pour décrire scRAW dans le rapport. Dans le code, ce preset est un alias du preset `default`, lui-même ancré sur `stable_generalist_trial_0017`.

TABLE 7 – Hyperparamètres du preset `stable_default` de scRAW.

Bloc	Paramètre	Valeur	Rôle
Prétraitement	<code>n_top_genes</code>	2000	Nombre de gènes hautement variables conservés.
Prétraitement	<code>min_genes_per_cell</code>	200	Filtrage des cellules de mauvaise qualité.
Prétraitement	<code>max_genes_per_cell</code>	None	Pas de borne supérieure sur le nombre de gènes détectés.
Prétraitement	<code>min_cells_per_gene</code>	3	Filtrage des gènes très peu détectés.
Prétraitement	<code>target_sum</code>	20000	Normalisation des comptes par cellule.
Prétraitement	<code>scale_max_value</code>	10.0	Seuil appliqué après standardisation.
Prétraitement	<code>hvg_flavor</code>	<code>seurat</code>	Méthode de sélection des HVG.
Prétraitement	<code>hvg_strategy</code>	<code>train_only</code>	Sélection des HVG sur les données d'entraînement.
Prétraitement	<code>dropout_method</code>	<code>none</code>	Pas de correction spécifique des zéros.
Prétraitement	<code>noise_level</code>	0.0	Pas de bruit ajouté au prétraitement.
Architecture	<code>input_type</code>	<code>processed</code>	Entrée utilisée par l'autoencodeur.
Architecture	<code>hidden_layers</code>	512, 256, 128	Tailles des couches cachées de l'encodeur.
Architecture	<code>z_dim</code>	256	Dimension de l'espace latent.
Architecture	<code>dropout</code>	0.3	Dropout dans l'encodeur et le décodeur.
Entraînement	<code>epochs</code>	120	Nombre total d'époques.
Entraînement	<code>warmup_epochs</code>	55	Phase sans pondération dynamique.
Entraînement	<code>lr</code>	0.00164076083297036	Taux d'apprentissage initial.
Entraînement	<code>batch_size</code>	192	Taille des mini-batches.
Entraînement	<code>seed</code>	64	Graine utilisée pour l'entraînement.
Entraînement	<code>random_state</code>	64	Graine de compatibilité pour les étapes non neurales.
Reconstruction	<code>reconstruction_distribution</code>	<code>mse</code>	Perte de reconstruction utilisée.
Reconstruction	<code>masking_rate</code>	0.1	Fraction d'entrées masquées aléatoirement.
Reconstruction	<code>masking_value</code>	0.0	Valeur utilisée pour les entrées masquées.
Reconstruction	<code>masked_recon_weight</code>	0.8	Poids interne des positions masquées.
Reconstruction	<code>masking_apply_weighted</code>	<code>True</code>	Masquage aussi actif pendant la phase pondérée.
Reconstruction	<code>nb_input_transform</code>	<code>log1p</code>	Paramètre conservé pour le mode NB, non central ici car la loss est MSE.
Reconstruction	<code>nb_theta</code>	19.237616208909103	Paramètre NB conservé par compatibilité.
Reconstruction	<code>nb_mu_clip_max</code>	1000000.0	Borne de sécurité du mode NB.

Suite page suivante

Bloc	Paramètre	Valeur	Rôle
Pondération	weight_exponent	0.2	Intensité de la pondération par taille de pseudo-cluster.
Pondération	density_knn_k	15	Voisinage utilisé pour estimer la densité locale.
Pondération	density_weight_exponent	1.0	Exposant de la composante densité.
Pondération	density_weight_clip	3.0	Borne supérieure avant normalisation de la densité.
Pondération	weight_fusion_mode	multiplicative	Fusion des composantes cluster et densité.
Pondération	cluster_weight_power	1.0	Puissance appliquée à la composante cluster.
Pondération	density_weight_power	1.0	Puissance appliquée à la composante densité.
Pondération	cluster_density_alpha	0.3483603718613933	Paramètre de fusion additive, conservé mais non utilisé par la fusion multiplicative.
Pondération	dynamic_weight_momentum	0.6884621079434989	Lissage des poids entre deux mises à jour.
Pondération	dynamic_weight_update_interval	20	Fréquence de recalcul des poids.
Pondération	min_cell_weight	0.3845423008053828	Borne inférieure du poids cellulaire final.
Pondération	max_cell_weight	10.0	Borne supérieure du poids cellulaire final.
Pseudo-labels	pseudo_label_method	leiden	Méthode utilisée pour les pseudo-clusters.
Pseudo-labels	pseudo_leiden_resolution_strategy	scraw_default	Recherche de résolution Leiden utilisée par scRAW.
Pseudo-labels	pseudo_leiden_target_clusters	0	Pas de cible explicite imposée.
Pseudo-labels	n_clusters	0	Pas de nombre de clusters fixé manuellement.
Pseudo-labels	unsupervised_k_fallback	0	Pas de fallback manuel de K .
Pseudo-labels	unsupervised_k_selection	heuristic	Estimation heuristique de K si nécessaire.
Pseudo-labels	unsupervised_k_min	8	Borne minimale de K .
Pseudo-labels	unsupervised_k_max	30	Borne maximale de K .
Pseudo-labels	unsupervised_k_num_candidates	12	Nombre maximal de candidats K .
Pseudo-labels	unsupervised_k_pca_dim	32	Dimension PCA éventuelle pour choisir K .
Pseudo-labels	unsupervised_k_eval_sample_size	3000	Taille d'échantillon pour les scores internes.
Pseudo-labels	unsupervised_k_stability_runs	5	Nombre de répétitions pour la stabilité.
Pseudo-labels	unsupervised_k_stability_sample_size	10000	Taille d'échantillon pour la stabilité.
Pseudo-labels	unsupervised_k_weight_stability	0.45	Poids du score de stabilité.
Pseudo-labels	unsupervised_k_weight_silhouette	0.25	Poids du score de silhouette.
Pseudo-labels	unsupervised_k_weight_ch	0.2	Poids du score Calinski-Harabasz.
Pseudo-labels	unsupervised_k_weight_db	0.1	Poids du score Davies-Bouldin.
Pseudo-labels	unsupervised_k_weight_tiny_clusters	0.25	Pénalité des très petits clusters.
Pseudo-labels	unsupervised_k_min_cluster_fraction	0.005	Seuil définissant un très petit cluster.
Pseudo-labels	unsupervised_k_overseg_penalty	0.25	Pénalité de sur-segmentation.
Pseudo-labels	unsupervised_k_underseg_penalty	0.05	Pénalité de sous-segmentation.
Triplet	rare_loss_type	triplet	Type de perte rare-aware.

Suite page suivante

Bloc	Paramètre	Valeur	Rôle
Triplet	rare_triplet_weight	0.05007581780188212	Poids de la triplet loss.
Triplet	rare_triplet_start_epoch	60	Début de la triplet loss.
Triplet	rare_triplet_margin	0.4	Marge de séparation.
Triplet	rare_triplet_min_weight	1.2	Poids minimal pour choisir une ancre.
Triplet	max_triplet_anchors_per_batch	64	Nombre maximal d'ancres par mini-batch.
Batch	use_batch_conditioning	True	Activation de la branche batch si plusieurs lots existent.
Batch	batch_correction_key	batch	Colonne de métadonnées utilisée pour le batch.
Batch	adversarial_batch_weight	0.11763398875166495	Poids de la perte adversariale batch.
Batch	adversarial_lambda	1.0	Intensité de l'inversion de gradient.
Batch	adversarial_start_epoch	30	Début de la branche adversariale.
Batch	adversarial_ramp_epochs	30	Durée de montée progressive du signal adversarial.
Batch	mmd_batch_weight	0.0	Terme MMD batch désactivé.
Clustering final	clustering_method	hdbscan	Méthode de clustering finale.
Clustering final	hdbscan_min_cluster_size	8	Taille minimale d'un cluster HDBSCAN.
Clustering final	hdbscan_min_samples	6	Nombre minimal d'échantillons HDBSCAN.
Clustering final	hdbscan_cluster_selection_method	random	Stratégie de sélection des clusters.
Clustering final	hdbscan_reassign_noise	False	Pas de réassignation des points bruit.
Clustering final	final_target_k	0	Pas de nombre final de clusters imposé.
Clustering final	hdbscan_scan_enabled	False	Pas de grille finale HDBSCAN.
Clustering final	hdbscan_scan_eval_sample_size	3000	Paramètre conservé pour compatibilité.
Clustering final	hdbscan_scan_include_alternative_method	True	Paramètre conservé pour compatibilité.
Suivi	capture_embedding_snapshots	False	Pas de capture systématique d'embeddings intermédiaires.
Suivi	snapshot_interval_epochs	10	Intervalle théorique de snapshots si activés.

E Figures de recherche d'hyperparamètres scRAW

Cette annexe regroupe les sorties Optuna d'importance moyenne des hyperparamètres pour les trois recherches utilisées dans la construction de `stable_default`.

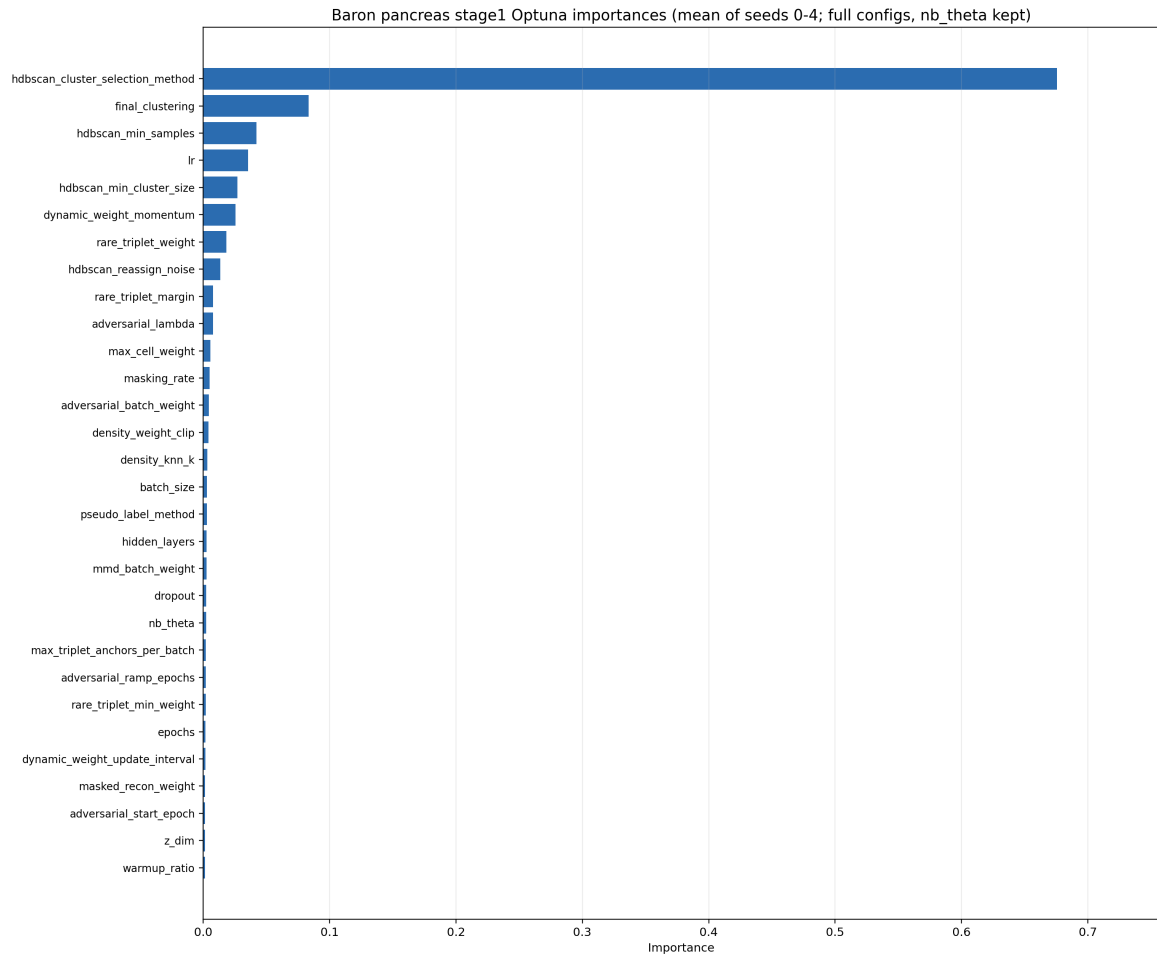


FIGURE 6 – Importance des hyperparamètres pour la recherche large sur Baron Human Pancreas.

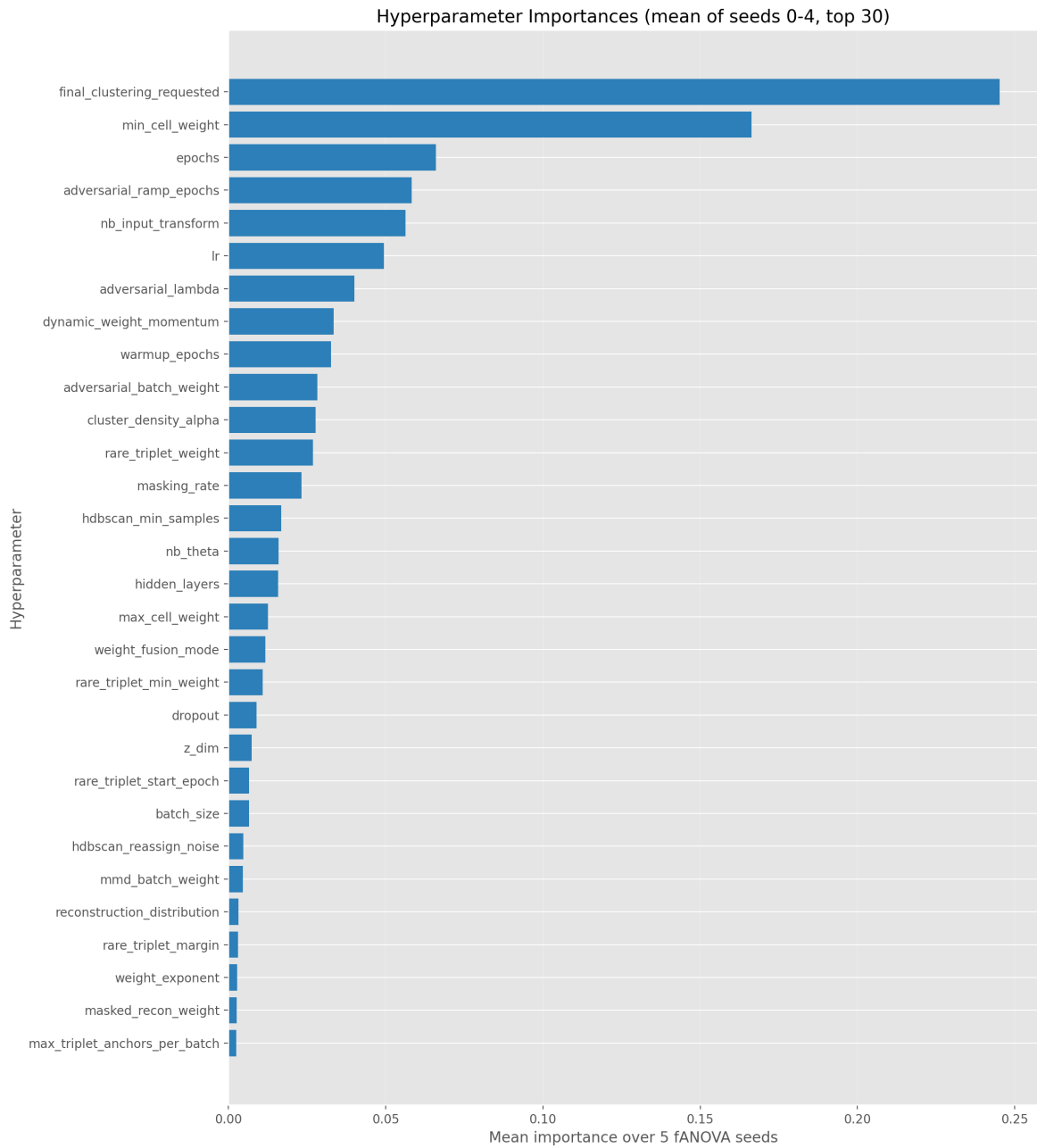
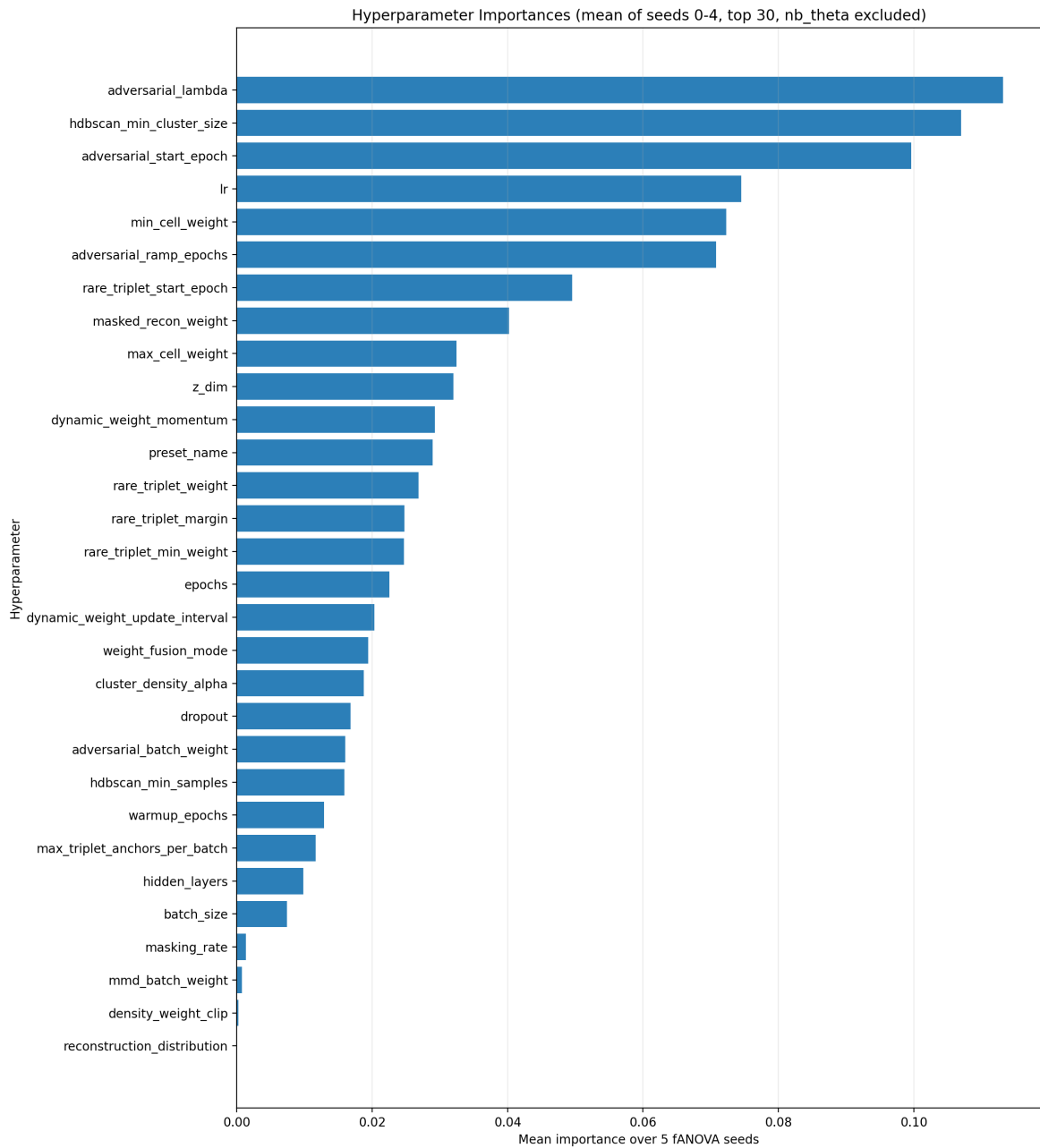


FIGURE 7 – Importance des hyperparamètres pour la recherche généraliste sur huit jeux de données.

FIGURE 8 – Importance des hyperparamètres pour la recherche restreinte `stable_generalist`.

F Screenshots de l'interface graphique SCRBenchmark

Cette section regroupe les captures d'écran illustrant le fonctionnement et le flux complet de l'interface utilisateur Streamlit développée pour SCRBenchmark.

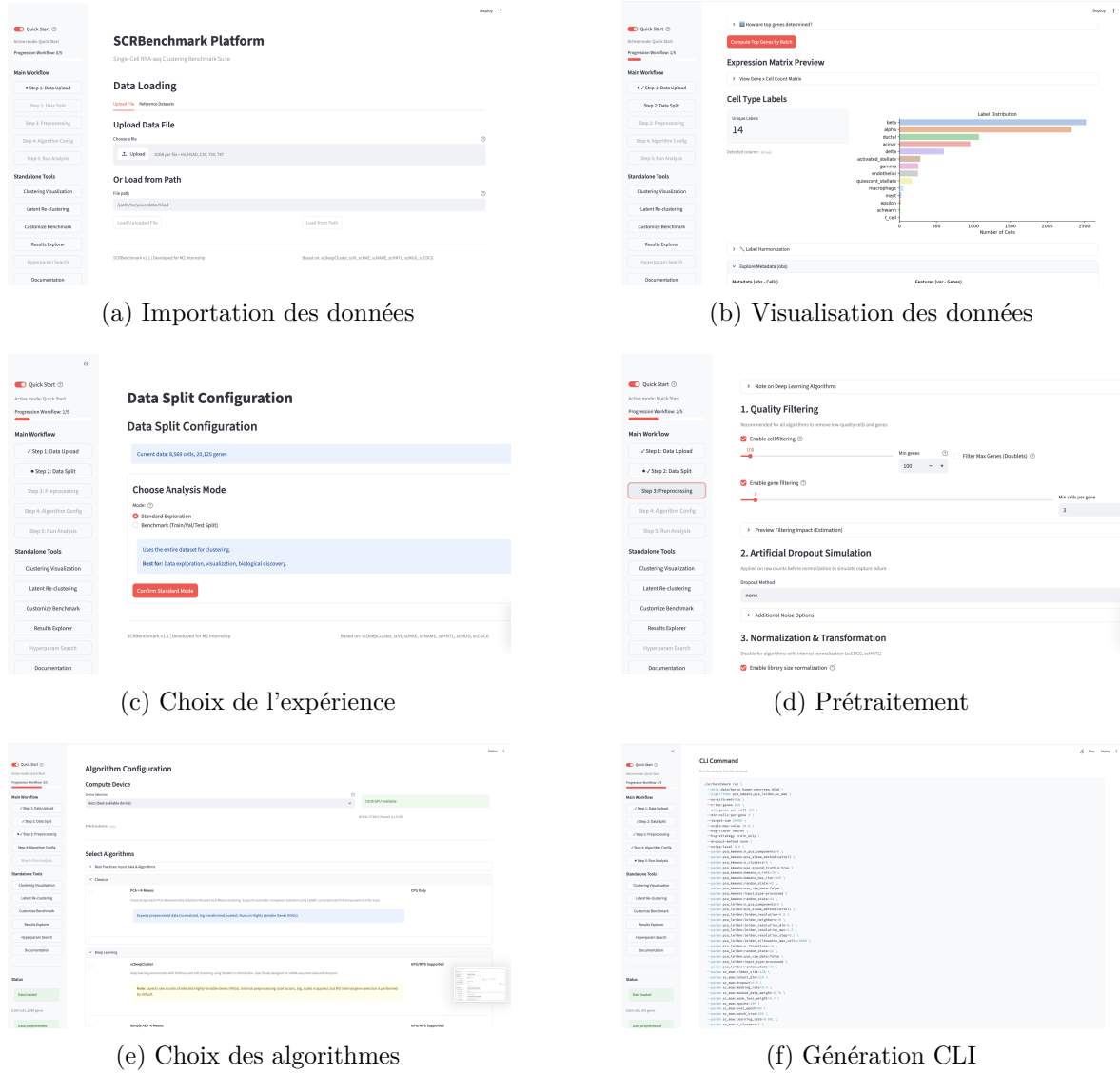
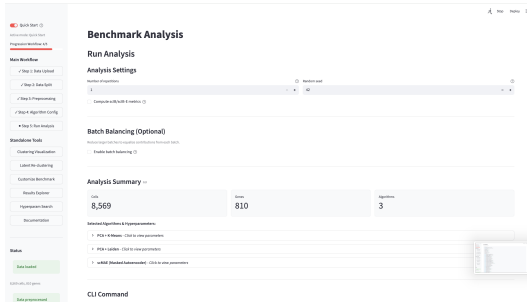


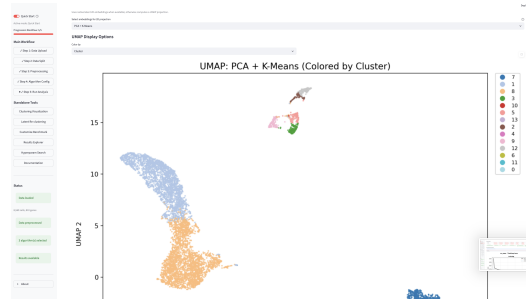
FIGURE 9 – Aperçu des différentes étapes de l'interface Streamlit de SCRBenchmark (1^{re} partie : configuration et préparation).



(g) Exécution des calculs



(h) Résultats des métriques



(i) UMAPs des résultats

FIGURE 9 – Aperçu des différentes étapes de l'interface Streamlit de SCRBenchmark (2^e partie : exécution et résultats).

Bibliographie

- [1] Dominic GRÜN et al. « Single-cell messenger RNA sequencing reveals rare intestinal cell types ». In : *Nature* 525.7568 (2015), p. 251-255. DOI : [10.1038/nature14966](https://doi.org/10.1038/nature14966).
- [2] Li LI et al. « Single-Cell RNA-Seq Analysis Maps Development of Human Germline Cells and Gonadal Niche Interactions ». In : *Cell Stem Cell* 20.6 (2017), 858-873.e4. DOI : [10.1016/j.stem.2017.03.007](https://doi.org/10.1016/j.stem.2017.03.007).
- [3] Yijie ZHANG et al. « Single-cell RNA sequencing in cancer research ». In : *Journal of Experimental & Clinical Cancer Research* 40.1 (2021), p. 81. DOI : [10.1186/s13046-021-01874-1](https://doi.org/10.1186/s13046-021-01874-1).
- [4] Atlas M. SARDOO et al. « Decoding brain memory formation by single-cell RNA sequencing ». In : *Briefings in Bioinformatics* 23.6 (2022), bbac412. DOI : [10.1093/bib/bbac412](https://doi.org/10.1093/bib/bbac412).
- [5] Hansruedi MATHYS et al. « Single-cell transcriptomic analysis of Alzheimer's disease ». In : *Nature* 570.7761 (2019), p. 332-337. DOI : [10.1038/s41586-019-1195-2](https://doi.org/10.1038/s41586-019-1195-2).
- [6] Malte D LUECKEN et Fabian J THEIS. « Current best practices in single-cell RNA-seq analysis : a tutorial ». In : *Molecular systems biology* 15.6 (2019), e8746.
- [7] F. Alexander WOLF, Philip ANGERER et Fabian J. THEIS. « Scanpy : large-scale single-cell gene expression data analysis ». In : *Genome Biology* 19.1 (2018), p. 15. DOI : [10.1186/s13059-017-1382-0](https://doi.org/10.1186/s13059-017-1382-0).
- [8] Vladimir Yu KISELEV, Tallulah S ANDREWS et Martin HEMBERG. « Challenges in unsupervised clustering of single-cell RNA-seq data ». In : *Nature Reviews Genetics* 20.5 (2019), p. 273-282. DOI : [10.1038/s41576-018-0088-9](https://doi.org/10.1038/s41576-018-0088-9).
- [9] Romain LOPEZ et al. « Deep generative modeling for single-cell transcriptomics ». In : *Nature Methods* 15.12 (2018), p. 1053-1058. DOI : [10.1038/s41592-018-0229-2](https://doi.org/10.1038/s41592-018-0229-2).
- [10] W Evan JOHNSON, Cheng LI et Ariel RABINOVIC. « Adjusting batch effects in microarray expression data using empirical Bayes methods ». In : *Biostatistics* 8.1 (2007), p. 118-127.
- [11] Ilya KORSUNSKY et al. « Fast, sensitive and accurate integration of single-cell data with Harmony ». In : *Nature methods* 16.12 (2019), p. 1289-1296.
- [12] Brian HIE, Bryan D. BRYSON et Bonnie BERGER. « Efficient integration of heterogeneous single-cell transcriptomes using Scanorama ». In : *Nature Biotechnology* 37.6 (2019), p. 685-691. DOI : [10.1038/s41587-019-0113-3](https://doi.org/10.1038/s41587-019-0113-3).
- [13] Yaroslav GANIN et al. « Domain-Adversarial Training of Neural Networks ». In : *Journal of Machine Learning Research* 17.59 (2016), p. 1-35.
- [14] Songwei GE et al. « Supervised Adversarial Alignment of Single-Cell RNA-seq Data ». In : *Journal of Computational Biology* 28.5 (2021), p. 501-513. DOI : [10.1089/cmb.2020.0439](https://doi.org/10.1089/cmb.2020.0439).

- [15] Qi ZHU et al. « Discriminative Domain Adaption Network for Simultaneously Removing Batch Effects and Annotating Cell Types in Single-Cell RNA-Seq ». In : *IEEE/ACM Transactions on Computational Biology and Bioinformatics* 21.6 (2024), p. 2543-2555. DOI : [10.1109/TCBB.2024.3487574](https://doi.org/10.1109/TCBB.2024.3487574).
- [16] Reut DANINO, Iftach NACHMAN et Roded SHARAN. « Batch correction of single-cell sequencing data via an autoencoder architecture ». In : *Bioinformatics Advances* 4.1 (2024), vbad186. DOI : [10.1093/bioadv/vbad186](https://doi.org/10.1093/bioadv/vbad186).
- [17] James MACQUEEN. « Some methods for classification and analysis of multivariate observations ». In : *Proceedings of the fifth Berkeley symposium on mathematical statistics and probability*. T. 1. 14. Oakland, CA, USA. 1967, p. 281-297.
- [18] Vincent D BLONDEL et al. « Fast unfolding of communities in large networks ». In : *Journal of Statistical Mechanics : Theory and Experiment* 2008.10 (2008), P10008.
- [19] Vincent A. TRAAG, Ludo WALTMAN et Nees Jan van ECK. « From Louvain to Leiden : guaranteeing well-connected communities ». In : *Scientific Reports* 9.1 (2019), p. 5233. DOI : [10.1038/s41598-019-41695-z](https://doi.org/10.1038/s41598-019-41695-z).
- [20] Zhaoyu FANG, Ruiqing ZHENG et Min LI. « scMAE : a masked autoencoder for single-cell RNA-seq clustering ». In : *Bioinformatics* 40.1 (2024), btae020. DOI : [10.1093/bioinformatics/btae020](https://doi.org/10.1093/bioinformatics/btae020).
- [21] Xiangjie LI et al. « Deep learning enables accurate clustering with batch effect removal in single-cell RNA-seq analysis ». In : *Nature Communications* 11.1 (2020), p. 2338.
- [22] Tian TIAN et al. « Clustering single-cell RNA-seq data with a model-based deep learning approach ». In : *Nature Machine Intelligence* 1.4 (2019), p. 191-198.
- [23] Hui WAN, Liang CHEN et Minghua DENG. « scNAME : neighborhood contrastive clustering with ancillary mask estimation for scRNA-seq data ». In : *Bioinformatics* 38.6 (2022), p. 1575-1583. DOI : [10.1093/bioinformatics/btac011](https://doi.org/10.1093/bioinformatics/btac011).
- [24] Karl PEARSON. « On lines and planes of closest fit to systems of points in space ». In : *Philosophical Magazine* 2.11 (1901), p. 559-572. DOI : [10.1080/14786440109462720](https://doi.org/10.1080/14786440109462720).
- [25] Ping XU et al. « scCDCG : Efficient Deep Structural Clustering for Single-Cell RNA-Seq via Deep Cut-Informed Graph Embedding ». In : *International Conference on Database Systems for Advanced Applications*. T. 14611. Lecture Notes in Computer Science. Springer, 2024, p. 195-210. DOI : [10.1007/978-981-97-5575-2_11](https://doi.org/10.1007/978-981-97-5575-2_11).
- [26] Lan JIANG et al. « GiniClust : detecting rare cell types from single-cell gene expression data with Gini index ». In : *Genome Biology* 17.1 (2016), p. 144. DOI : [10.1186/s13059-016-1010-4](https://doi.org/10.1186/s13059-016-1010-4).
- [27] Marilisa NERI et al. « CellSIUS provides sensitive and specific detection of rare cell populations from complex single cell RNA-seq data ». In : *Genome Biology* 20.142 (2019), p. 142. DOI : [10.1186/s13059-019-1748-0](https://doi.org/10.1186/s13059-019-1748-0).
- [28] J. LIU et al. « scAIDE : clustering of large-scale single-cell RNA-seq data reveals putative and rare cell types ». In : *NAR Genomics and Bioinformatics* 2.4 (2020), lqaa082. DOI : [10.1093/nargab/lqaa082](https://doi.org/10.1093/nargab/lqaa082).

- [29] Tianyuan LEI et al. « Self-supervised deep clustering of single-cell RNA-seq data to hierarchically detect rare cell populations ». In : *Briefings in Bioinformatics* 24.5 (2023), bbad335. DOI : [10.1093/bib/bbad335](https://doi.org/10.1093/bib/bbad335).
- [30] Yunpei XU et al. « scCAD : Cluster decomposition-based anomaly detection for rare cell identification in single-cell expression data ». In : *Nature Communications* 15.1 (2024), p. 7561. DOI : [10.1038/s41467-024-51891-9](https://doi.org/10.1038/s41467-024-51891-9).
- [31] Ping XU et al. « scCluBench : Comprehensive Benchmarking of Clustering Algorithms for Single-Cell RNA Sequencing ». In : *Proceedings of the AAAI Conference on Artificial Intelligence*. T. 40. 2026, p. 1364-1372.
- [32] Maayan BARON et al. « A single-cell transcriptomic map of the human and mouse pancreas reveals inter- and intra-cell population structure ». In : *Cell Systems* 3.4 (2016), 346-360.e4. DOI : [10.1016/j.cels.2016.08.011](https://doi.org/10.1016/j.cels.2016.08.011).
- [33] Junyuan XIE, Ross GIRSHICK et Ali FARHADI. « Unsupervised deep embedding for clustering analysis ». In : *International Conference on Machine Learning*. PMLR. 2016, p. 478-487.
- [34] Xifeng GUO et al. « Improved deep embedded clustering on images ». In : *Proceedings of the International Joint Conference on Neural Networks*. 2017, p. 2430-2437.
- [35] Mohammad LOTFOLLAHI et al. « Query-to-reference single-cell integration with transfer learning ». In : *Nature Biotechnology* 39.11 (2021), p. 1422-1434.
- [36] Harold W. KUHN. « The Hungarian Method for the Assignment Problem ». In : *Naval Research Logistics Quarterly* 2.1-2 (1955), p. 83-97. DOI : [10.1002/nav.3800020109](https://doi.org/10.1002/nav.3800020109).
- [37] Takuya AKIBA et al. « Optuna : A Next-generation Hyperparameter Optimization Framework ». In : *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2019, p. 2623-2631.
- [38] Ricardo J. G. B. CAMPELLO, Davoud MOULAVI et Jörg SANDER. « Hierarchical Density Estimates for Data Clustering, Visualization, and Outlier Detection ». In : *ACM Transactions on Knowledge Discovery from Data* 10.1 (2015), p. 1-51. DOI : [10.1145/2733381](https://doi.org/10.1145/2733381).
- [39] Diederik P. KINGMA et Jimmy BA. « Adam : A Method for Stochastic Optimization ». In : *International Conference on Learning Representations (ICLR)*. 2015.
- [40] Isaac VIRSHUP et al. « anndata : Access and store annotated data matrices ». In : *Journal of Open Source Software* 9.101 (2024), p. 4371. DOI : [10.21105/joss.04371](https://doi.org/10.21105/joss.04371).
- [41] Changhu WANG, Zihao CHEN et Ruibin XI. « Feature screening for clustering analysis of count data with an application to single-cell RNA-sequencing ». In : *The Annals of Applied Statistics* 19.4 (2025), p. 2738-2758. DOI : [10.1214/25-AOAS2102](https://doi.org/10.1214/25-AOAS2102).
- [42] Mathilde CARON et al. « Deep Clustering for Unsupervised Learning of Visual Features ». In : *Proceedings of the European Conference on Computer Vision (ECCV)*. 2018, p. 132-149.

- [43] Jing WANG et al. « scSCCNIA : similarity matrix based contrastive clustering with neighbor information aggregation for single-cell RNA sequencing data ». In : *Briefings in Bioinformatics* 27.2 (2026), bbag094. DOI : [10.1093/bib/bbag094](https://doi.org/10.1093/bib/bbag094).
- [44] Hua MENG, Chuan QIN et Zhiguo LONG. « scHNTL : single-cell RNA-seq data clustering augmented by high-order neighbors and triplet loss ». In : *Bioinformatics* 41.2 (2025), btaf044. DOI : [10.1093/bioinformatics/btaf044](https://doi.org/10.1093/bioinformatics/btaf044).
- [45] De-Min LIANG et Pu-Feng DU. « scMUG : deep clustering analysis of single-cell RNA-seq data on multiple gene functional modules ». In : *Briefings in Bioinformatics* 26.2 (2025), bbaf138. DOI : [10.1093/bib/bbaf138](https://doi.org/10.1093/bib/bbaf138).
- [46] Fatemeh S. Hashemi GOLPAYEGANI et al. « Interpretable Self-Supervised Prototype Learning for Single-Cell Transcriptomics ». In : *ICLR Workshop on Learning Meaningful Representations of Life (LMRL)*. 2025.
- [47] Chenxin YI et al. « Benchmarking deep learning methods for biologically conserved single-cell integration ». In : *Genome Biology* 26.1 (2025), p. 398. DOI : [10.1186/s13059-025-03869-z](https://doi.org/10.1186/s13059-025-03869-z).
- [48] Malte D. LUECKEN et al. « Benchmarking atlas-level data integration in single-cell genomics ». In : *Nature Methods* 19.1 (2022), p. 41-50. DOI : [10.1038/s41592-021-01336-8](https://doi.org/10.1038/s41592-021-01336-8).
- [49] Yushan QIU et al. « scTPC : a novel semisupervised deep clustering model for scRNA-seq data ». In : *Bioinformatics* 40.5 (avr. 2024), btae293. DOI : [10.1093/bioinformatics/btae293](https://doi.org/10.1093/bioinformatics/btae293).
- [50] OPENPROBLEMS. *Pancreas Dataset (v1)*. https://openproblems.bio/datasets/openproblems_v1/pancreas. 2026.